

1.INTRODUCTION

The modern hospitality and food service industry is undergoing a profound paradigm shift driven by consumer demand for health-conscious dining, real-time dietary transparency, and contactless digital interactions. Traditional dining establishments rely predominantly on static, physical paper or laminated cardboard menus. While familiar, physical menus present acute operational limitations: they are expensive to reprint whenever prices or ingredients change, they cannot dynamically reflect immediate kitchen item availability, they cannot verify nutritional values, and they cannot calculate individual ingredient customizations in real time.

Healthy Bite is conceptualized and engineered to bridge this operational divide. Built as a high-performance, responsive multi-tenant web application, Healthy Bite empowers seated restaurant guests to scan a table-side cryptographic QR code using any smartphone browser, instantly gaining access to a rich digital culinary catalog. Without installing native applications or undergoing cumbersome registration flows, customers can browse dishes, filter by dietary lifestyles (Vegetarian, Non-Vegetarian, Vegan, Jain), inspect potential allergens, customize dish portions and add-ons, view dynamic 8-macro nutritional calculations, and transmit verified orders directly to the kitchen.

2. OBJECTIVES

2.1 Project Overview

Healthy Bite is a lightweight, object-oriented **PHP MVC restaurant management system** that connects customers, kitchen staff, restaurant owners, and the Super Admin.

- **Customer:** Browse QR menu, customize food, view nutrition, place orders, pay, and track orders.
- **Kitchen Staff:** Manage orders using a 5-stage Kanban board.
- **Restaurant Owner:** Manage menu, tables, QR codes, staff, reviews, and sales analytics.
- **Super Admin:** Manage and monitor restaurants and users across the platform.

2.2 Objectives of the System

The main objectives are to:

- Replace paper menus with **digital QR-based menus**.
- Provide **8 nutrition details** for customized food items.
- Validate prices, GST, and nutrition **on the server** to prevent tampering.
- Improve kitchen order handling through a **Kanban system**.
- Secure table access using **unique QR tokens**.
- Support **multiple restaurants** with separate management.

2.3 Scope of the System

In-Scope

QR menu access, restaurant-specific menus, food customization, dynamic price and nutrition calculation, cart and checkout, simulated Cash/UPI/Card payment, order tracking, kitchen Kanban, table and QR management, reviews, staff management, analytics, and Super Admin control.

Out-of-Scope

Live payment gateway integration, inventory automation, native mobile apps, WebSocket servers, and cloud receipt-printer integration.

2.4 Major Features

1. Secure QR-based table access
2. Digital food catalog with dietary and allergen information
3. Food customization with portion and add-ons
4. Dynamic **8-macro nutrition calculation**
5. Server-verified price and **5% GST** calculation
6. Cart and checkout system
7. Frozen order snapshots
8. Cash, UPI, and Card payment simulation
9. 5-stage kitchen Kanban
10. Live customer order tracking
11. Table and QR management
12. Customer reviews and owner replies
13. Sales and nutrition analytics
14. Multi-restaurant Super Admin governance

2.5 Target Users / System Actors

- **Dining Customer:** Browses menu, customizes food, orders, pays, tracks orders, and reviews.
- **Kitchen Staff:** Views and updates customer orders.
- **Restaurant Owner/Manager:** Manages menu, tables, staff, reviews, and analytics.
- **Super Admin:** Manages restaurants, users, and platform-level operations.

3.SYSTEM ANALYSIS

3.1 Problem Definition

In conventional restaurant operations, the traditional ordering process relies on physical paper or cardboard menus and verbal order communication. During peak lunch and dinner hours, this manual workflow introduces severe friction: customers experience long wait times before a waiter arrives to take their order; dietary preferences and allergy requirements are frequently miscommunicated verbally to the kitchen; physical menus fail to reflect sold-out items, leading to customer disappointment; and health-conscious diners have zero visibility into calorie, protein, or sugar contents. Furthermore, printing physical menus represents a continuous recurring expense whenever prices or recipes are updated. Healthy Bite eliminates these operational inefficiencies by establishing a direct, digitized bridge between diners, kitchen expeditors, and restaurant management.

3.2 Existing System vs. Proposed System

Operational Parameter	Existing Traditional System	Proposed Healthy Bite Architecture
Ordering Mechanism	Waiters physically bring laminated paper cards; verbal dictation.	Contactless table QR code scanning; instant browser menu access.
Menu Updates & Pricing	Requires expensive physical reprints; prices can become outdated.	Real-time updates via Owner Portal; instant price/dish reflection.
Nutritional Transparency	Virtually non-existent; zero allergen or macro tracking.	Real-time 8-macro breakdown (Calories, Protein, Carbs, Fat, etc.).
Dish Customization	Prone to verbal miscommunication and kitchen errors.	Interactive UI modal; live price adjustments and portion scaling.
Kitchen Communication	Manual physical KOT paper transport to kitchen counter.	Real-time 5-column Kitchen Kanban board (Placed to Completed).
Live Order Tracking	Customers must wave down waitstaff to ask about food status.	Live 5-stage progress tracker polling real-time kitchen status.
Billing & Payment	Manual calculation by cashier; delay in bill presentation.	Server-verified bill computation with simulated payment settlement.
Hygiene & Contact	Physical menus handled by dozens of customers daily.	Zero physical contact; completely hygienic mobile interface.

FEASIBILITY STUDY

1. Technical Feasibility

The technical feasibility of Healthy Bite is exceptionally robust. The system is engineered using battle-tested web standards: PHP 8.2+ with strict typing, MySQL 8.x (InnoDB engine) with transactional foreign key integrity, Native PHP Data Objects (PDO) with parameterized prepared statements, Semantic HTML5, Vanilla CSS3 with custom variables, and Vanilla JavaScript (ES6+). The entire application runs seamlessly on standard LAMP/XAMPP server configurations and requires zero proprietary third-party runtime frameworks or licensed servers.

2. Economic Feasibility

From an economic standpoint, Healthy Bite represents a cost-effective solution with immediate operational return on investment (ROI). By eliminating paper menu design and periodic reprinting expenses, restaurants save thousands of rupees annually. The reliance on open-source technologies (PHP, MariaDB/MySQL, Vanilla JS) eliminates expensive software licensing fees. Furthermore, table turnaround efficiency increases by up to 25%, allowing restaurants to serve more guests during peak dining hours without hiring additional front-of-house staff.

3. Operational Feasibility

The operational feasibility of the system is substantiated by its intuitive, consumer-grade user experience design. Dining customers require no training whatsoever; scanning a QR code is a universally understood interaction across modern smartphone users. The kitchen Kanban board uses simple, high-contrast action buttons ('Accept Order', 'Start Preparing', 'Mark Ready', 'Complete Order') designed specifically for fast-paced commercial kitchen environments.

4. Schedule Feasibility

The development lifecycle was planned and executed within the designated academic timeframe (June 2026 – October 2026), ensuring sufficient allocation for planning, analysis, architecture, coding, testing, release validation, and documentation.

4.SDLC

SDLC TIMELINE FOR HEALTHY BITE PROJECT (JUNE 15 – OCT 25)

1. Project Planning (June 15 – June 25) → 10 days

- Define the project problem and objectives.
- Identify project scope and limitations.
- Select the required technology stack.
- Prepare the feasibility study.

2. Requirement Analysis (June 26 – July 5) → 10 days

- Collect system requirements.
- Prepare the SRS document.
- Identify users, modules, and system features.
- Define dietary and nutrition requirements.

3. System Design (July 6 – July 20) → 15 days

- Create ER, DFD, and UML diagrams.
- Design the database schema.
- Prepare the system architecture and UI design.

4. Implementation / Coding (July 21 – Sept 10) → ~50 days

- Develop the custom PHP MVC architecture.
- Implement the PDO database layer.
- Develop the 8-macro nutrition and pricing system.
- Build customer, kitchen, owner, and admin modules.
- Implement the UI and core functionality.

5. Testing & Integration (Sept 11 – Sept 30) → 20 days

- Integrate all system modules.
- Perform functional and security testing.
- Test database operations and system validation.
- Fix errors and perform regression testing.

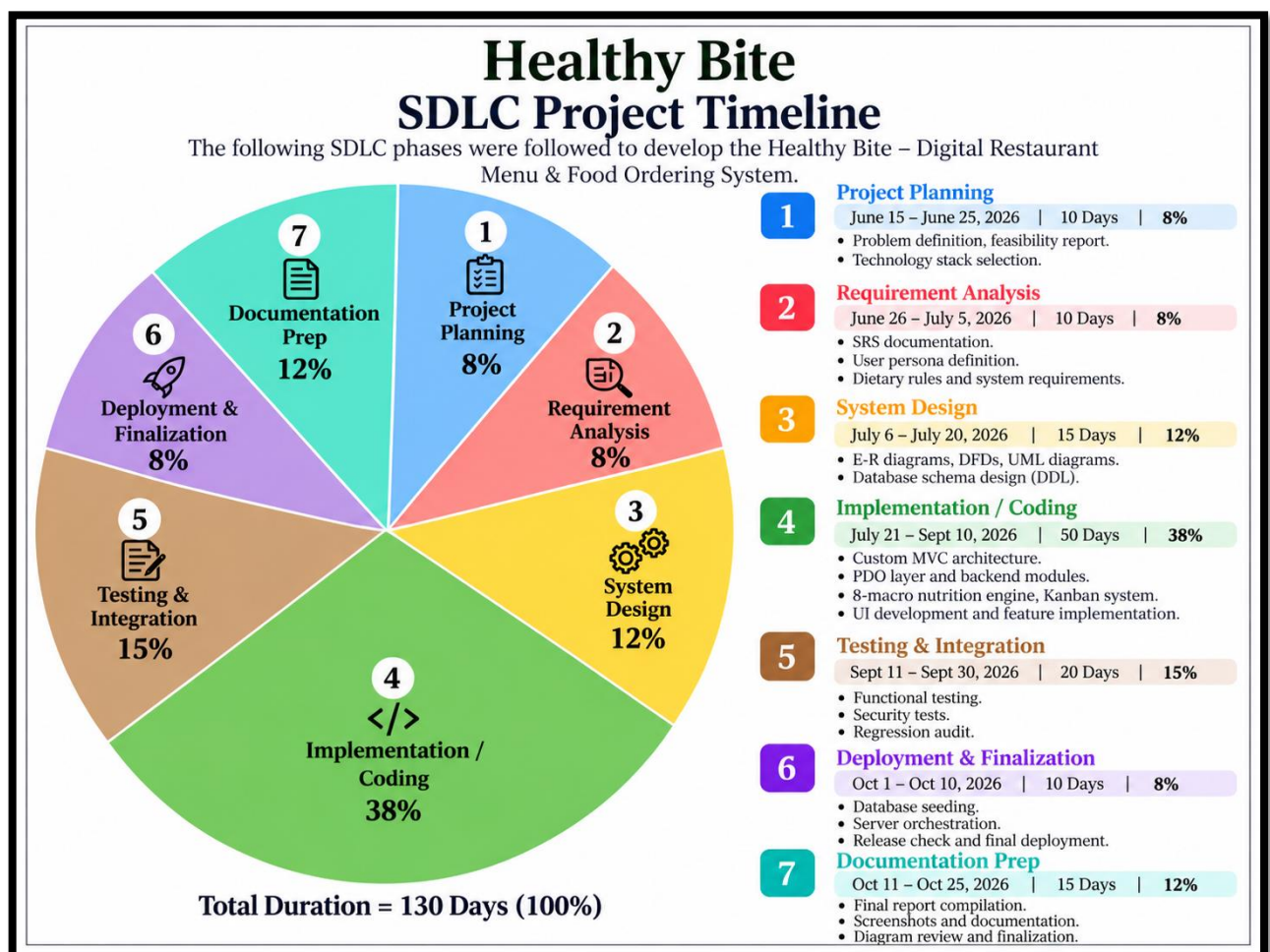
- Prepare the testing report.

6. Deployment & Finalization (Oct 1 – Oct 10) → 10 days

- Seed and configure the database.
- Set up the server/demo environment.
- Verify all major features.
- Perform the final release check.

7. Documentation Preparation (Oct 11 – Oct 25) → 15 days

- Complete the final project report.
- Add diagrams, screenshots, and testing results.
- Review and finalize project documentation.
- Prepare the PPT for viva and demonstration.
- Complete final submission.

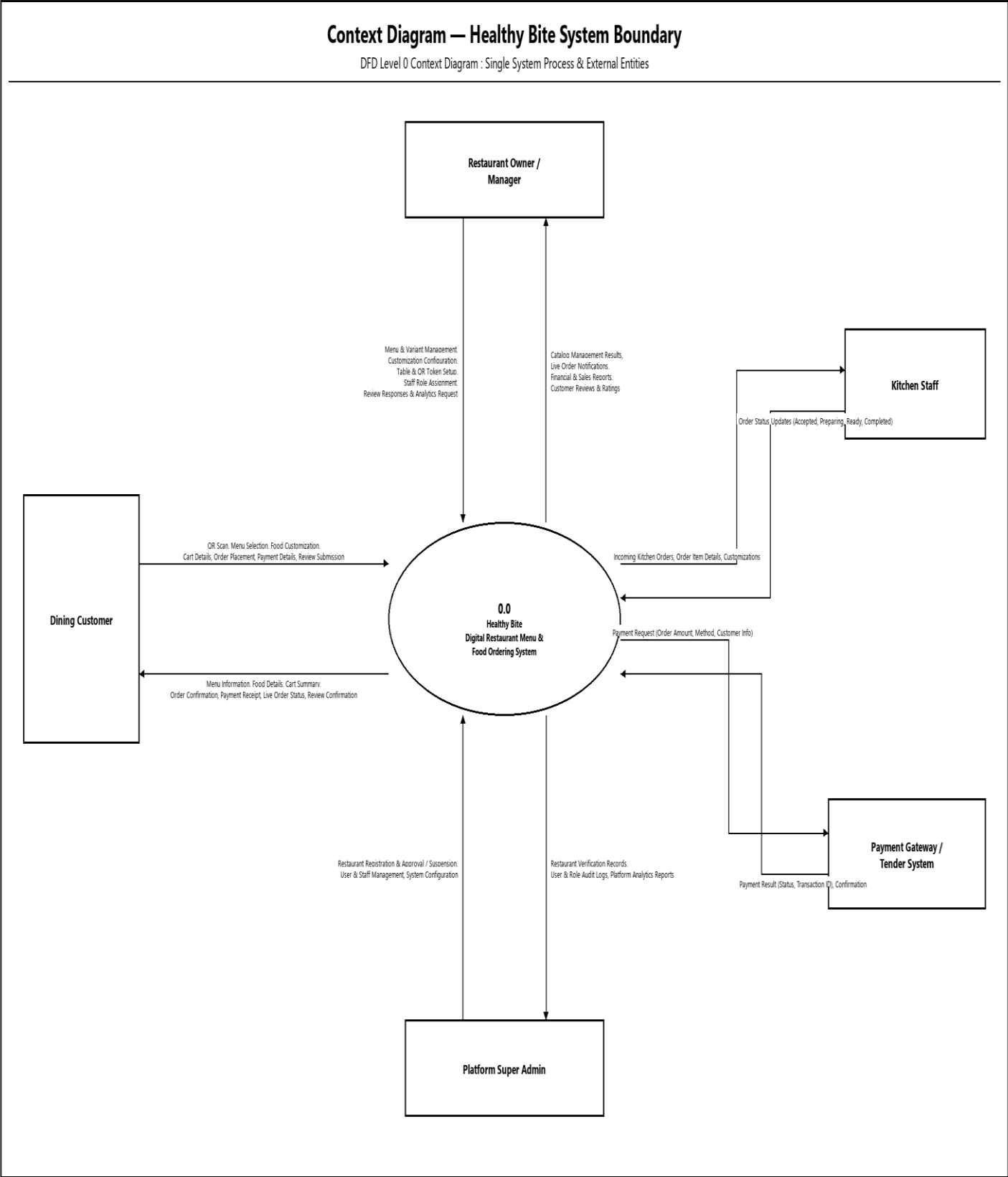


5. SYSTEM DESIGN

DFD

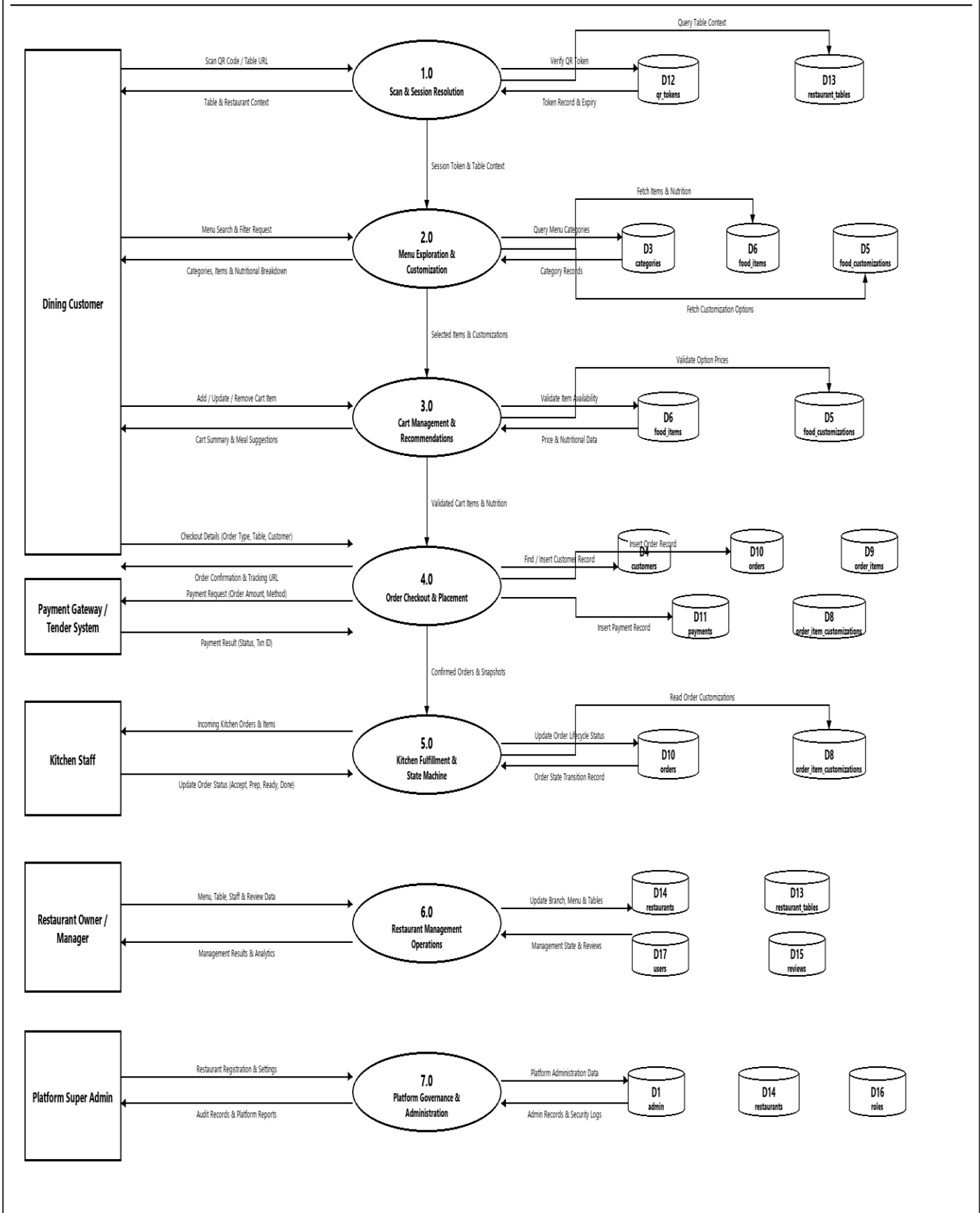
Context Diagram — Healthy Bite System Boundary

DFD Level 0 Context Diagram : Single System Process & External Entities



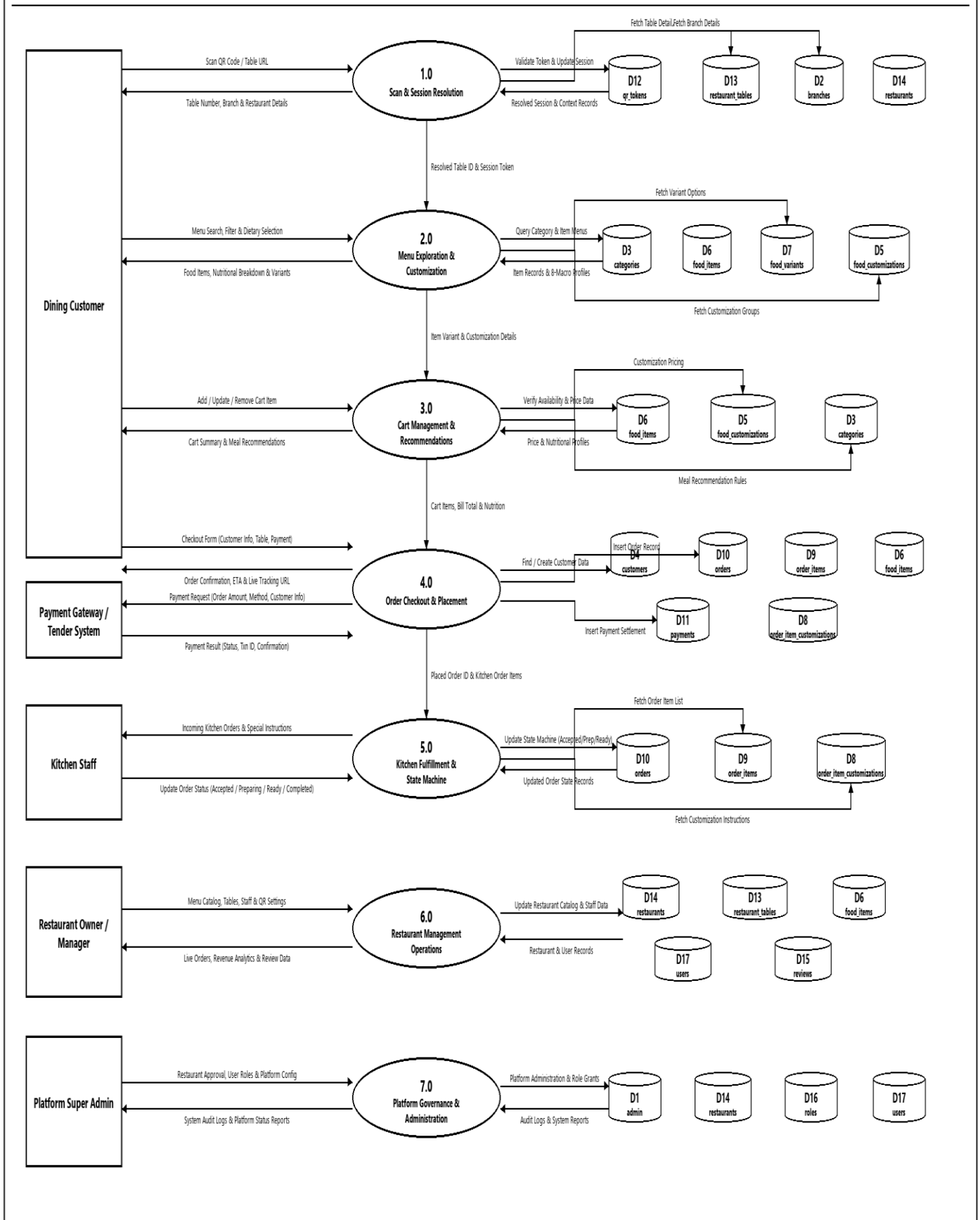
DFD Level 0 — System Functional Decomposition

Healthy Bite — High-Level Functional Modules & Entity Interfacing



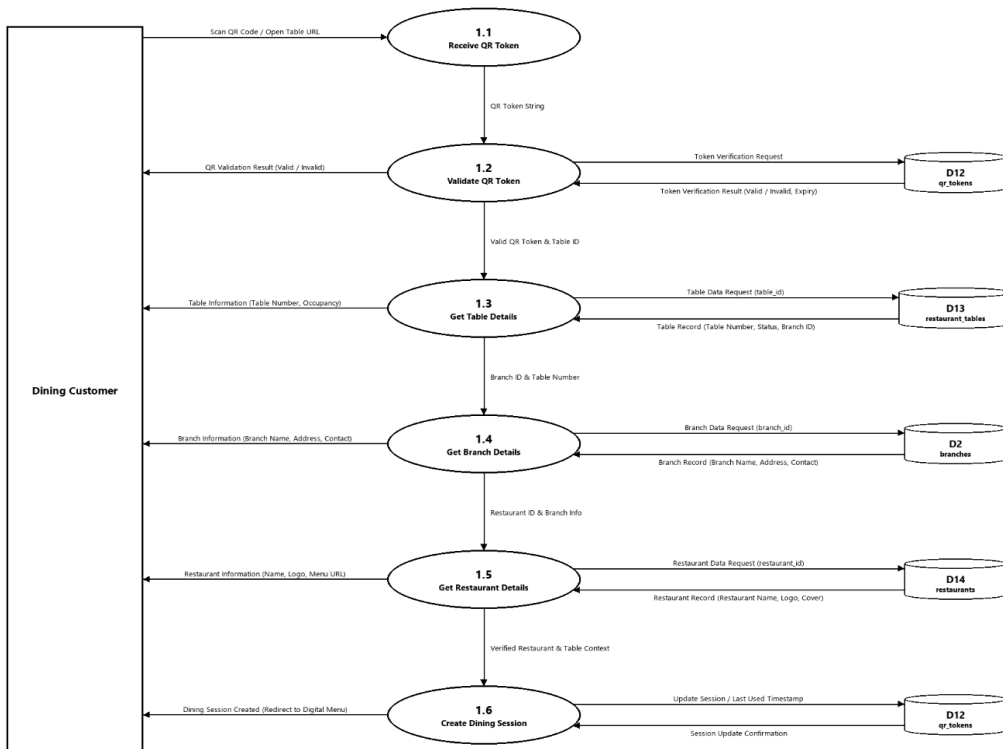
DFD Level 1 — Subsystem Operational Flow Diagram

Healthy Bite — Comprehensive Data Store Read/Write Balancing



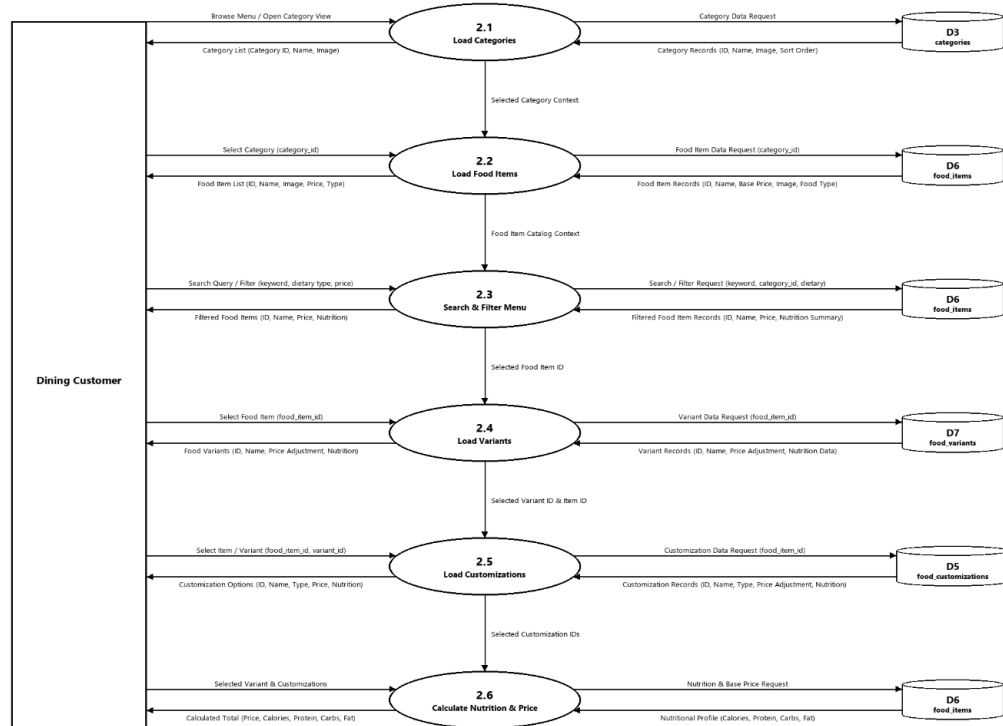
DFD - 2 : Level 2 (1.0 Scan & Session Resolution)

Subsystem Decomposition : Cryptographic QR Token Resolution & Dining Session Initialization



DFD - 3 : Level 2 (2.0 Menu Exploration & Customization)

Subsystem Decomposition : Menu Catalog, Multi-Variant Configuration & 8-Macro Engine



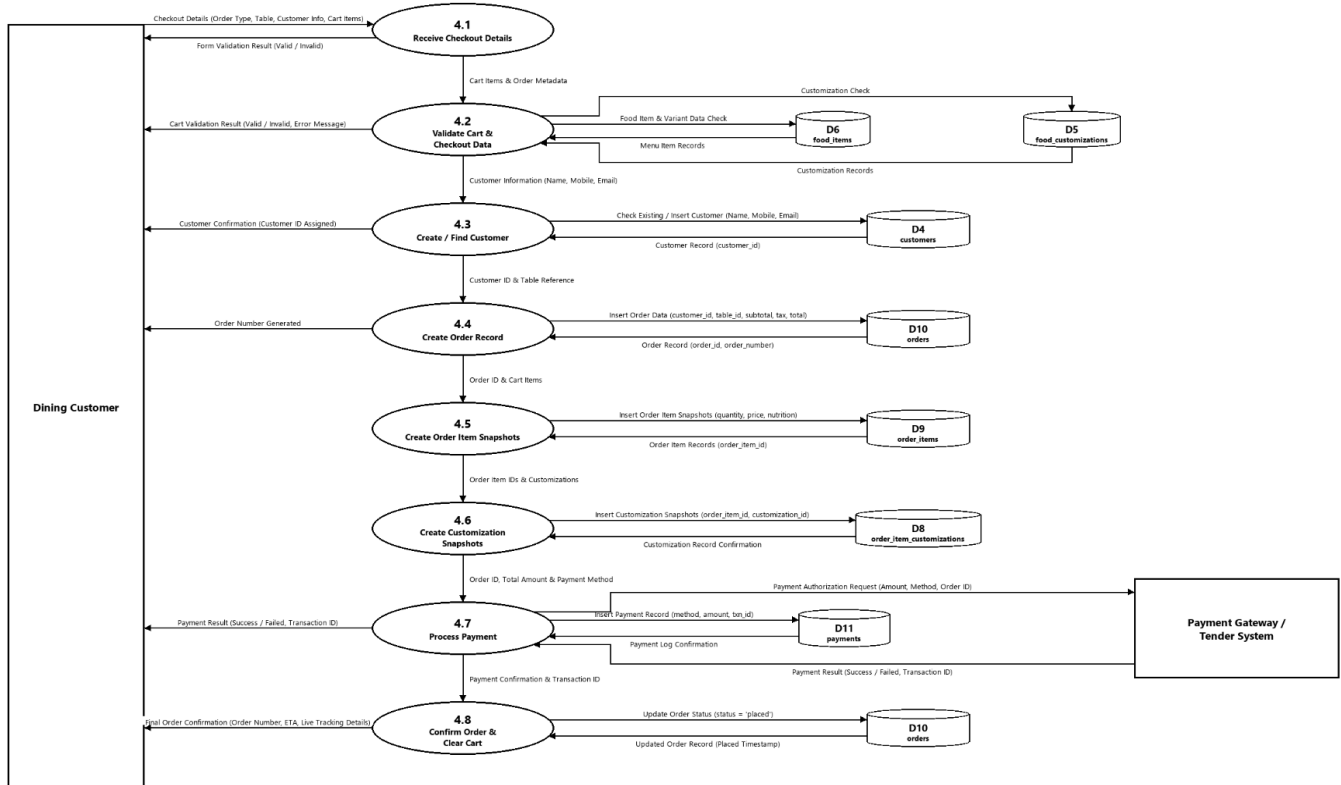
DFD - 4 : Level 2 (3.0 Cart Management & Recommendations)

Subsystem Decomposition : Client Cart State, Dynamic Totals & Recommendation Engine



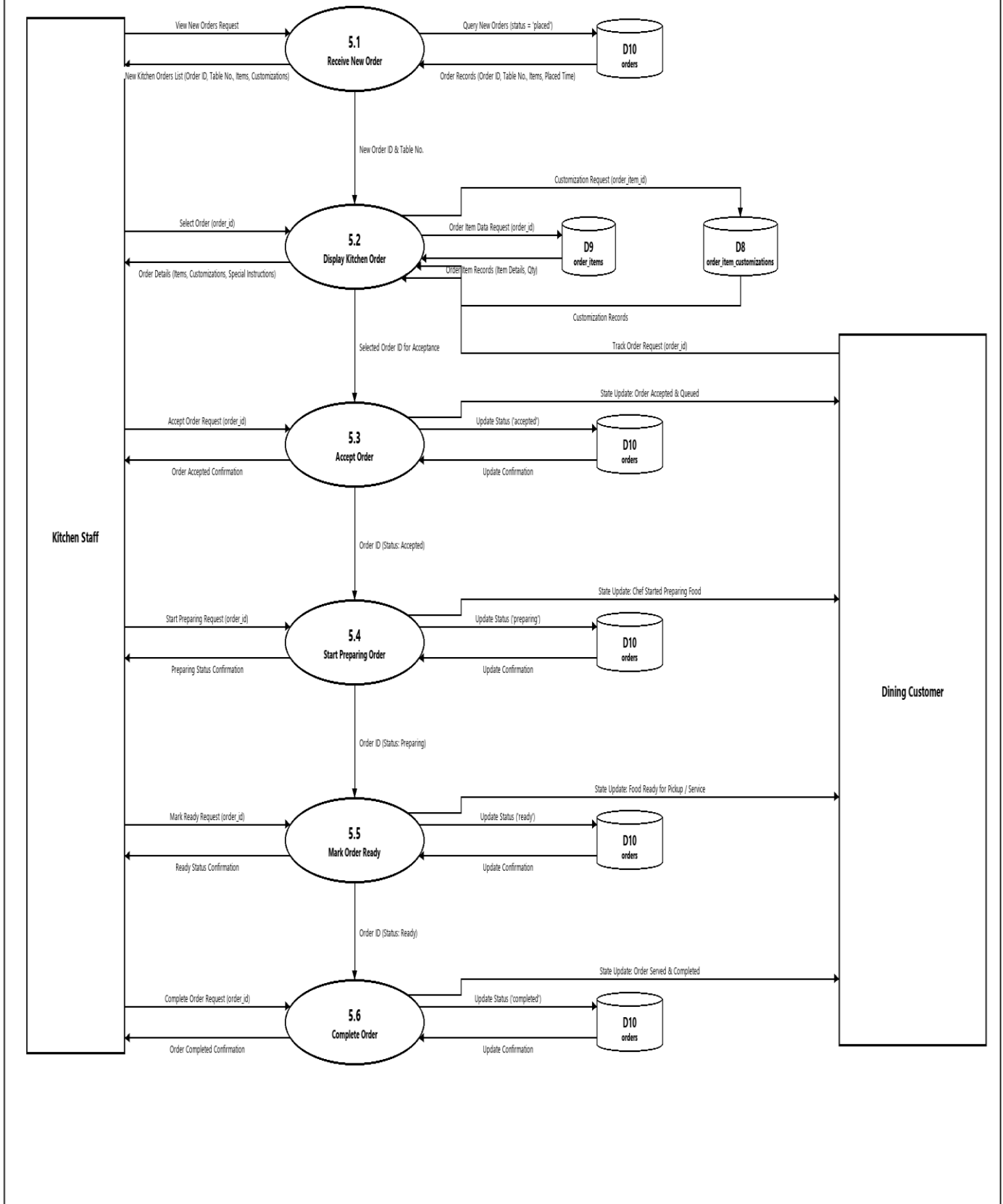
DFD - 5 : Level 2 (4.0 Order Checkout & Placement)

Subsystem Decomposition : Transactional Order Insertion, Snapshots & Payment Gateway Integration

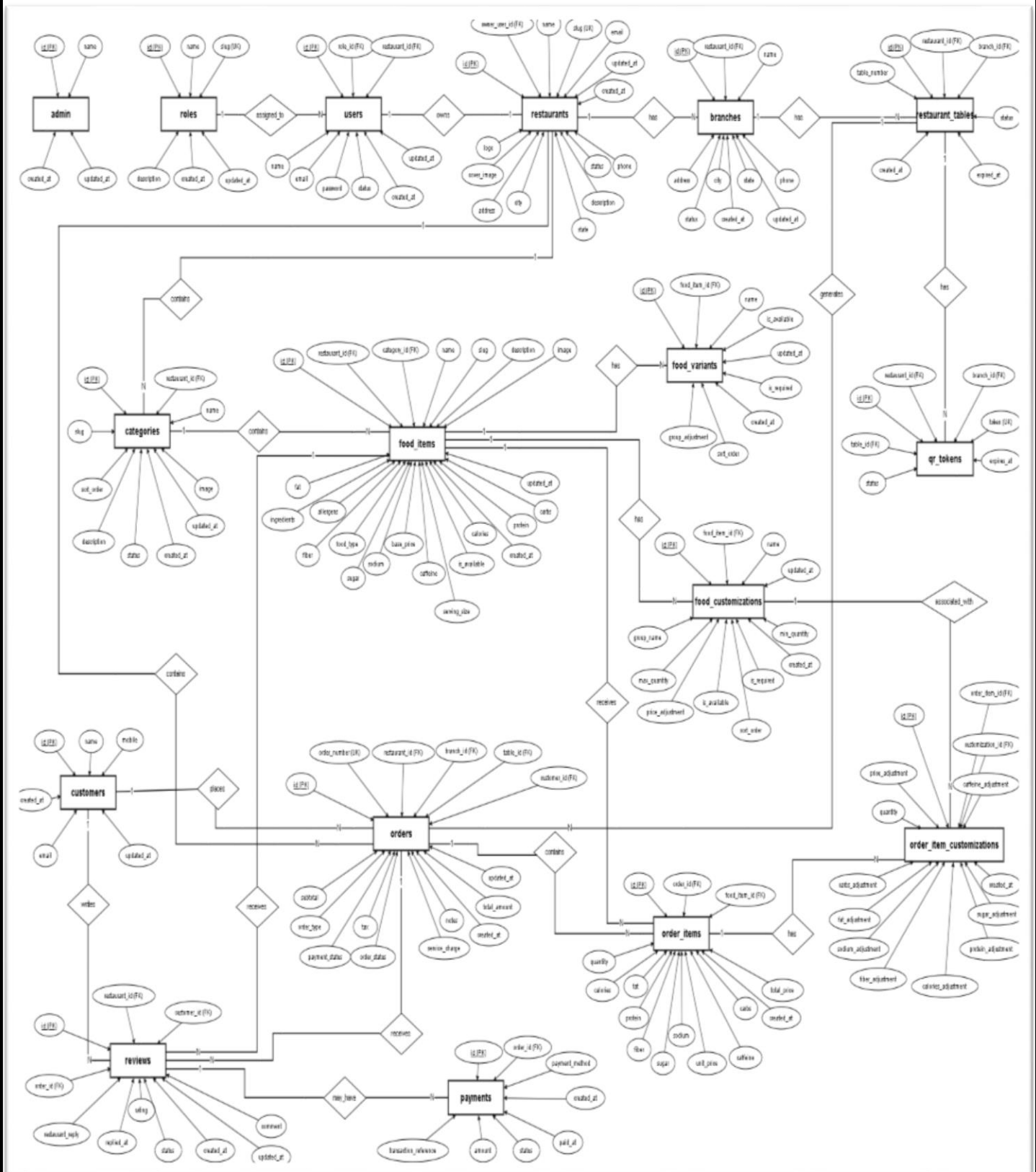


DFD - 6 : Level 2 (5.0 Kitchen Fulfillment & State Machine)

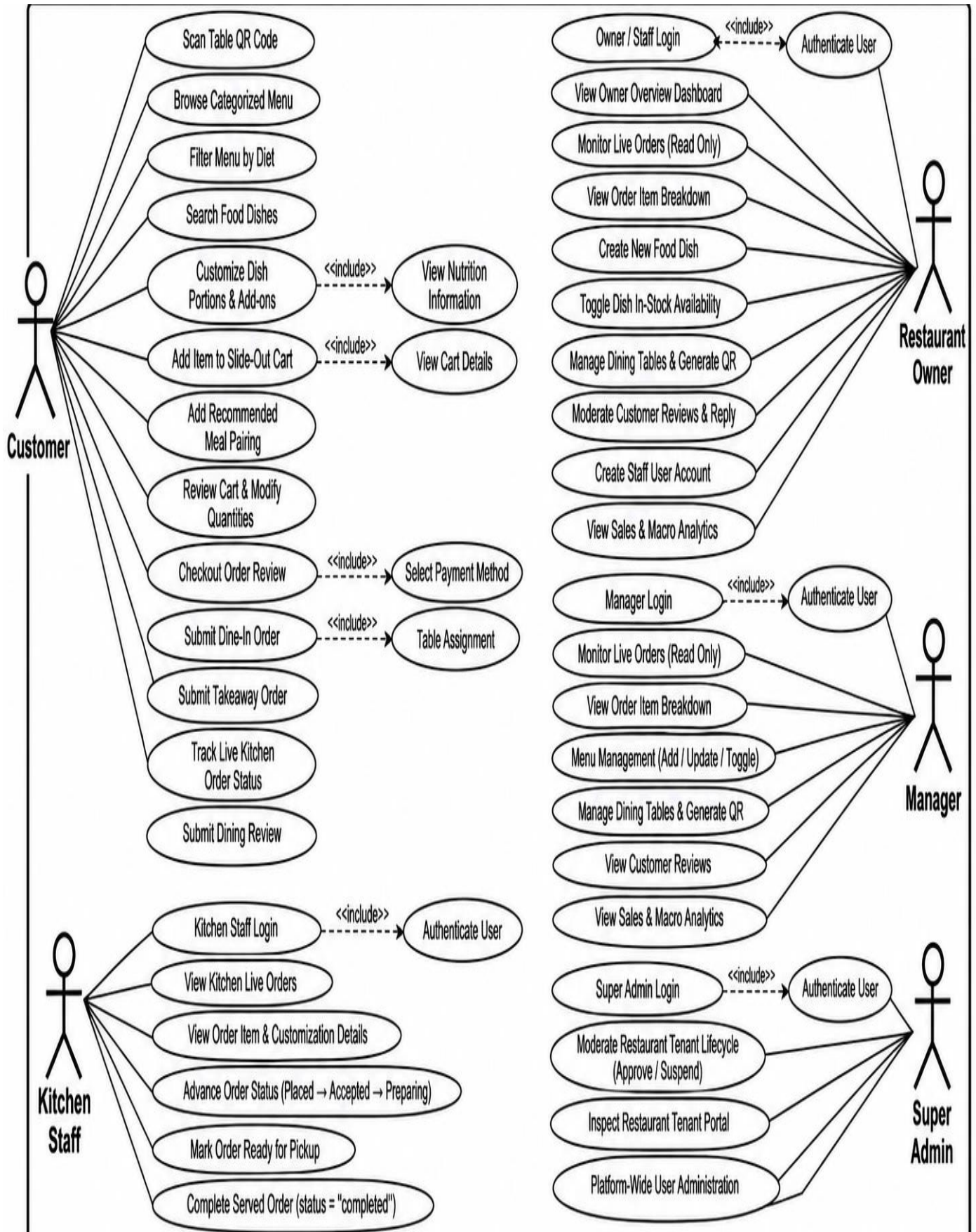
Subsystem Decomposition : Kitchen Kanban Lifecycle, State Transitions & Customer Notifications



Entity Relationship Diagram (ERD)



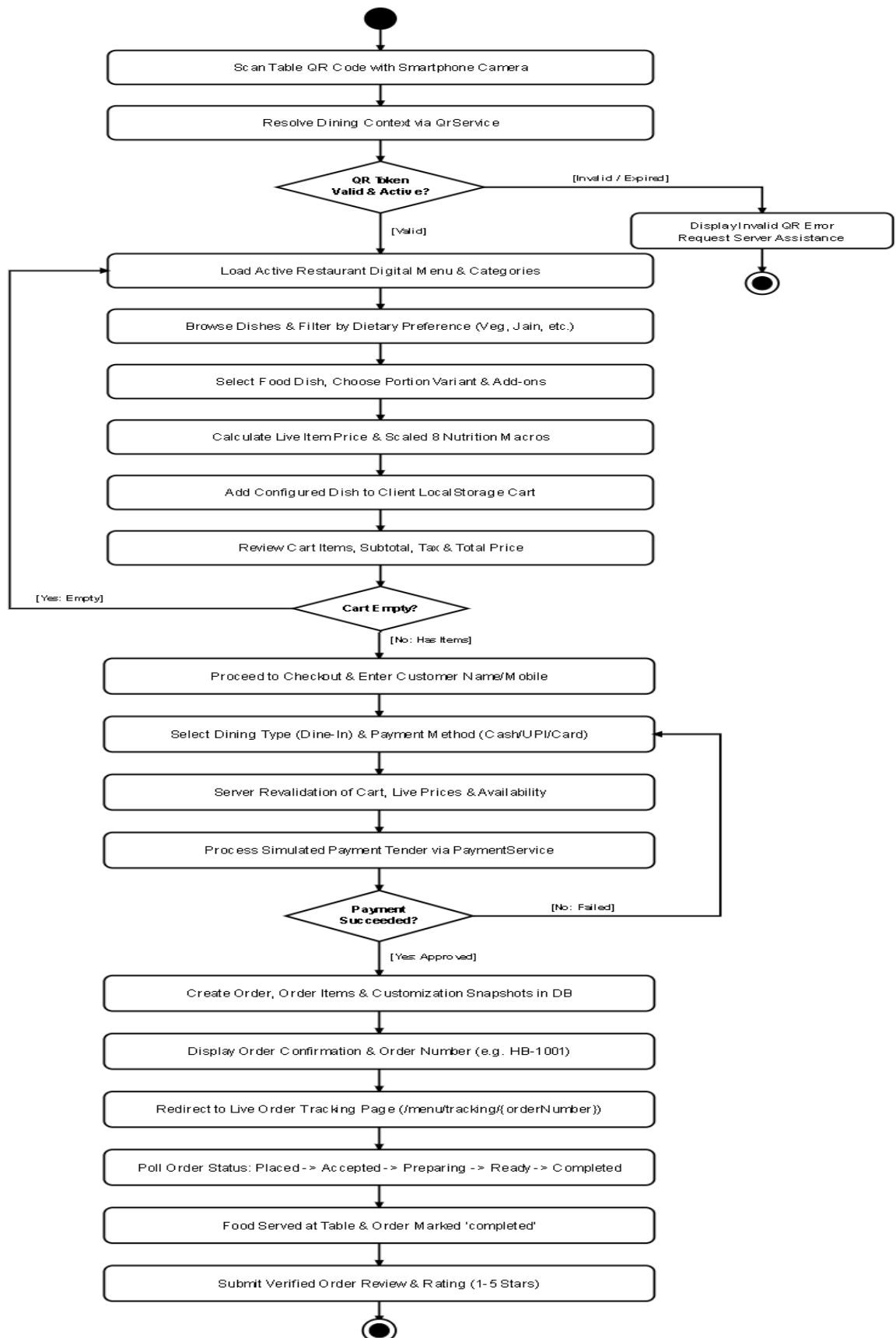
Use Case Diagram



Activity Diagram

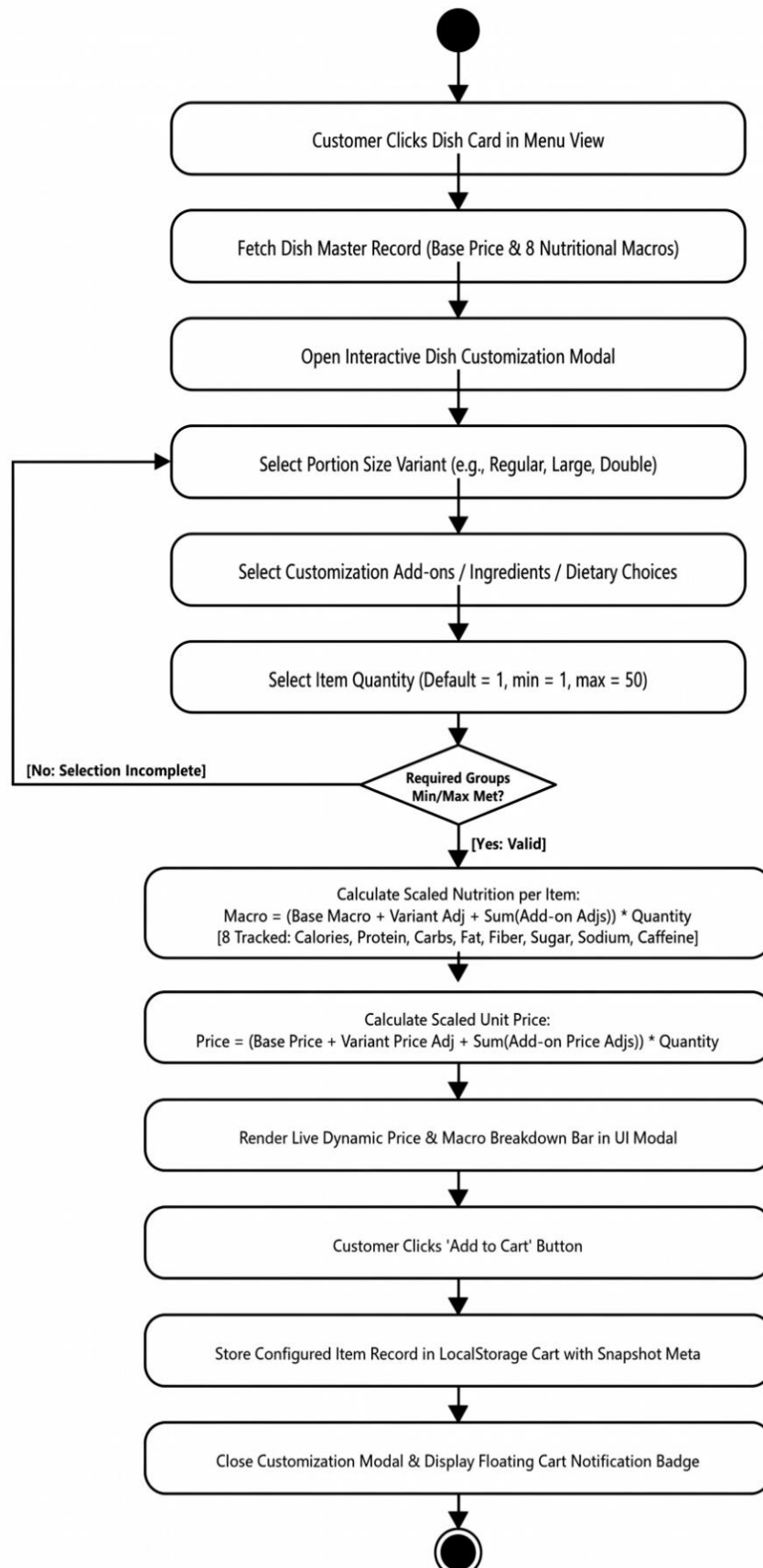
AD-01 : CUSTOMER ORDERING WORKFLOW

UML Activity Diagram • End-to-End Table QR Dining, Cart Validation, Tracking & Review



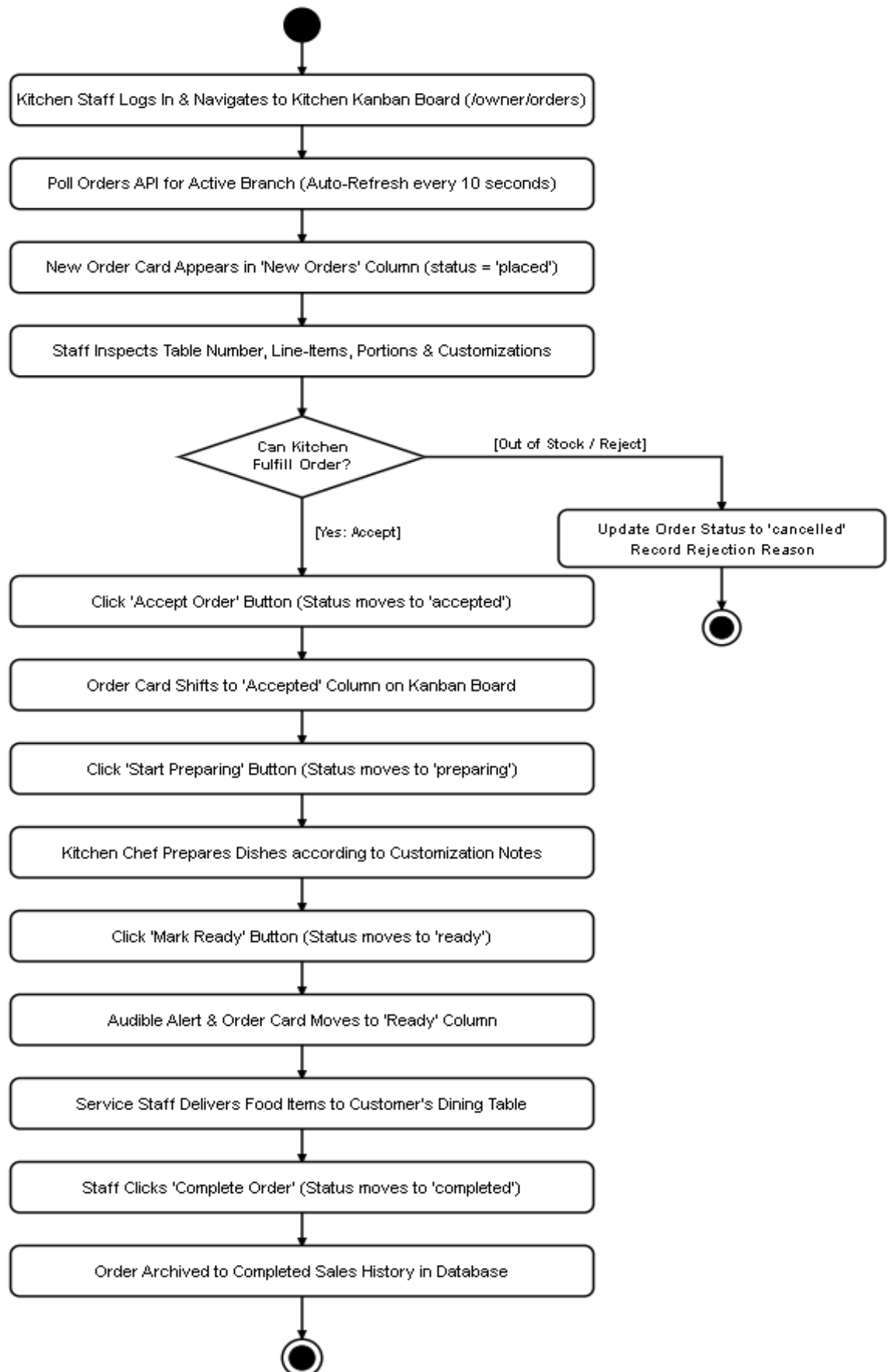
AD-02: FOOD CUSTOMIZATION & PRICE/NUTRITION CALCULATION

UML Activity Diagram • Variant Scaling, Customization Deltas & All 8 Nutritional Macros



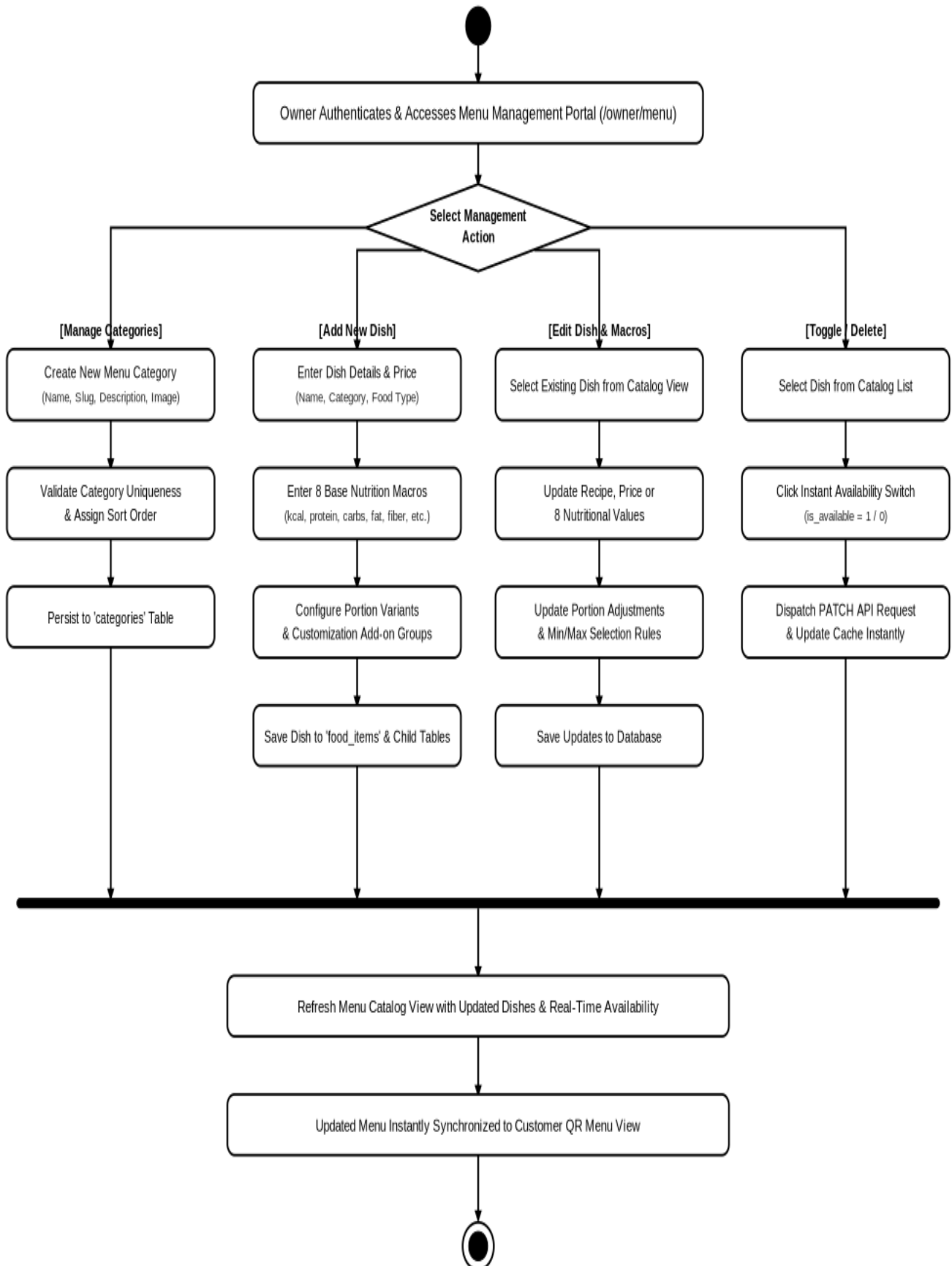
AD-03: KITCHEN LIVE ORDER MANAGEMENT

UML Activity Diagram • Kitchen Kanban Transitions (placed -> accepted -> preparing -> ready -> completed)



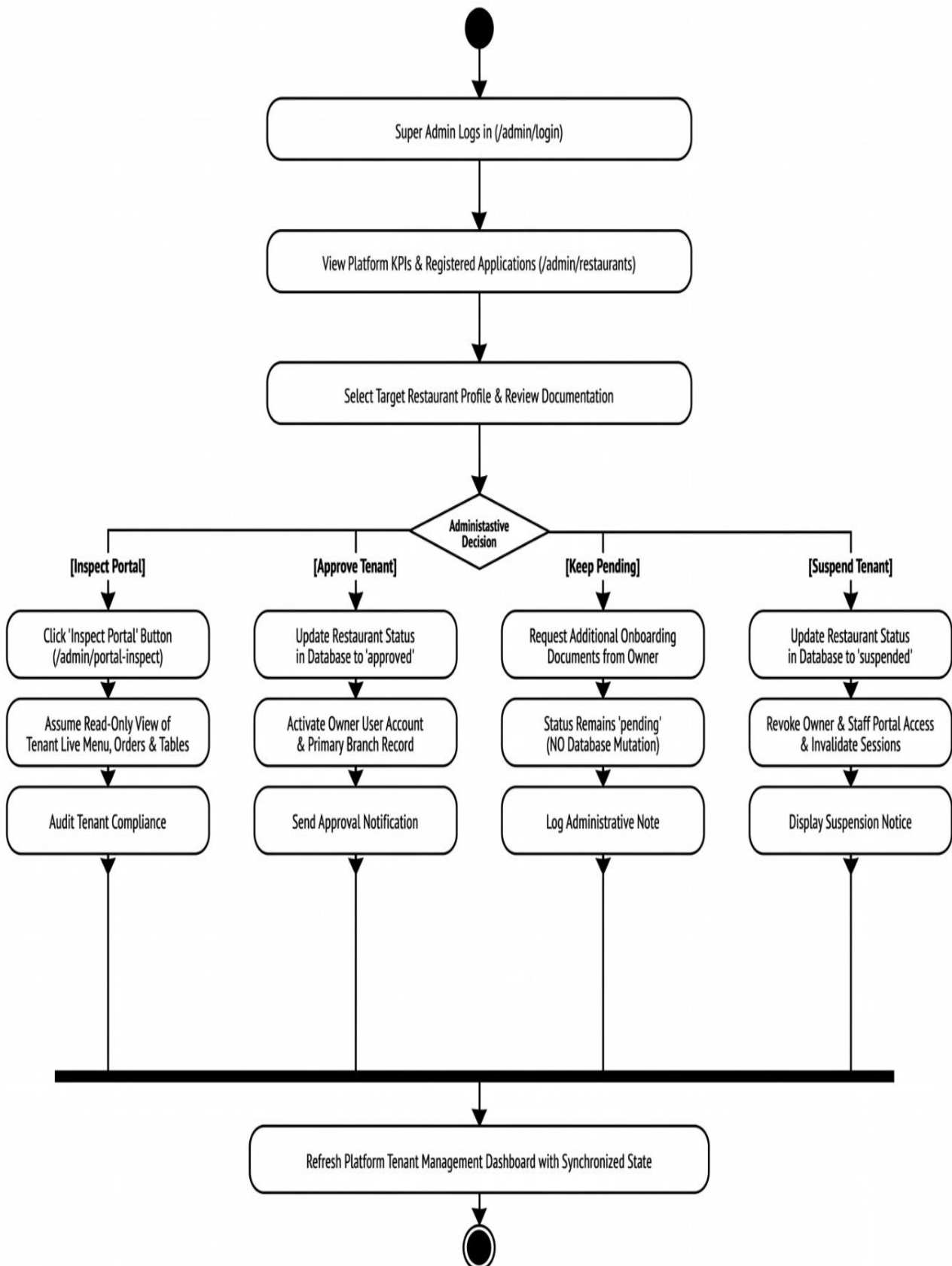
AD-04: RESTAURANT OWNER FOOD & MENU MANAGEMENT

UML Activity Diagram • Category CRUD, Dish Creation, 8 Macros, Portion Variants & Availability Toggles



AD-05: SUPER ADMIN RESTAURANT & PLATFORM MANAGEMENT

UML Activity Diagram • Multi-Tenant Onboarding, Approval, Portal Inspection & Suspension Lifecycle



6.DATA DICTIONARY

TABLE: ADMIN

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Internal unique identifier for legacy admin record; auto-incremented.
name	VARCHAR(120)	None	Full name of legacy system administrator.
created_at	TIMESTAMP	None	Date and time when the legacy admin record was initialized.
updated_at	TIMESTAMP	None	Timestamp of the last modification to the legacy admin record.

TABLE: ROLES

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; unique system authority role identifier.
name	VARCHAR(50)	None	Human-readable role title (e.g. Super Admin, Restaurant Owner, Manager, Staff).
slug	VARCHAR(50)	UK	Unique URL/machine slug identifier (super_admin, restaurant_owner, manager, staff).
description	VARCHAR(255)	None	Functional description of security privileges and portal access rights.
created_at	TIMESTAMP	None	Record creation timestamp (default CURRENT_TIMESTAMP).
updated_at	TIMESTAMP	None	Record last updated timestamp (default CURRENT_TIMESTAMP).

TABLE: RESTAURANTS

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; multi-tenant restaurant entity identifier.
owner_user_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing users(id); designates primary restaurant owner.
name	VARCHAR(150)	None	Legal trade name of the dining establishment (e.g. Greenhouse Kitchen).
slug	VARCHAR(150)	UK	Unique URL-friendly slug used for tenant resolution and branding.
logo	VARCHAR(255)	None	Relative path or URL to high-resolution brand logo image.
cover_image	VARCHAR(255)	None	Path or URL to restaurant hero cover banner graphic.
description	TEXT	None	Comprehensive brand narrative, culinary focus, and restaurant overview.
phone	VARCHAR(30)	None	Primary customer service and inquiry contact telephone number.
email	VARCHAR(191)	None	Official restaurant business contact email address.
address	TEXT	None	Physical street address of restaurant headquarters or primary facility.
city	VARCHAR(100)	None	Operating city where the restaurant facility is located.
state	VARCHAR(100)	None	Operating state or administrative province.
status	ENUM('pending','approved','suspended')	None	Platform tenancy status governed by Super Admin (default 'approved').
created_at	TIMESTAMP	None	Date and time when the restaurant was registered on the platform.

updated_at	TIMESTAMP	None	Timestamp of the most recent profile modification.
------------	-----------	------	--

TABLE: USERS

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; user account unique identifier.
role_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing roles(id); defines user authorization level.
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); scopes user to tenant (NULL for Super Admin).
name	VARCHAR(120)	None	Full legal name of the user or staff employee.
email	VARCHAR(191)	UK	Unique email address utilized for portal authentication.
password	VARCHAR(255)	None	Secure one-way BCrypt cryptographic password hash.
status	ENUM('active','inactive','suspended')	None	Operational account status (default 'active').
created_at	TIMESTAMP	None	User registration timestamp.
updated_at	TIMESTAMP	None	Account profile modification timestamp.

TABLE: BRANCHES

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; physical dining branch outlet identifier.
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); parent restaurant entity.
name	VARCHAR(150)	None	Physical branch title (e.g. Indiranagar Flagship, Koramangala Outlet).

address	TEXT	None	Physical street location of the specific branch outlet.
city	VARCHAR(100)	None	Operating city of the branch.
state	VARCHAR(100)	None	Operating state of the branch.
phone	VARCHAR(30)	None	Direct telephone contact for branch operations and reservations.
status	ENUM('active','inactive')	None	Operational state of the branch location (default 'active').
created_at	TIMESTAMP	None	Branch record creation timestamp.
updated_at	TIMESTAMP	None	Branch record modification timestamp.

TABLE: CATEGORIES

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; menu category identifier.
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); scopes category to restaurant.
name	VARCHAR(100)	None	Category title (e.g. Protein Bowls, Artisan Wraps, Superfood Salads, Soups).
slug	VARCHAR(100)	None	URL slug used for client-side category filtering.
description	TEXT	None	Detailed description of the food category classification.
image	VARCHAR(255)	None	Category icon or representative banner image path.
sort_order	INT(11)	None	Integer defining visual display order on customer menu (default 0).
status	ENUM('active','inactive')	None	Category display toggle state (default 'active').

created_at	TIMESTAMP	None	Category creation timestamp.
updated_at	TIMESTAMP	None	Category modification timestamp.

TABLE: FOOD_ITEMS

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; master food dish identifier.
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); owning restaurant tenant.
category_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing categories(id); menu classification.
name	VARCHAR(150)	None	Culinary dish display title (e.g. Grilled Chicken & Quinoa Protein Bowl).
slug	VARCHAR(150)	None	URL-safe unique dish slug for routing and APIs.
description	TEXT	None	Comprehensive sensory and culinary description of the food item.
image	VARCHAR(255)	None	High-resolution Unsplash photo URL or local asset path.
ingredients	TEXT	None	List of primary ingredients and culinary preparation components.
allergens	VARCHAR(255)	None	Declared allergen warnings (e.g. Dairy, Tree Nuts, Gluten, Soy).
food_type	ENUM('vegetarian','non _vegetarian','vegan','jain ';other')	None	Dietary lifestyle classification badge.
base_price	DECIMAL(10,2)	None	Standard baseline menu price in Indian Rupees (₹).
calories	INT UNSIGNED	None	Total nutritional energy content measured in kilocalories (kcal).

protein	DECIMAL(6,2)	None	Total protein content measured strictly in grams (g).
carbs	DECIMAL(6,2)	None	Total carbohydrate content measured strictly in grams (g).
fat	DECIMAL(6,2)	None	Total dietary fat content measured strictly in grams (g).
fiber	DECIMAL(6,2)	None	Dietary fiber content measured strictly in grams (g).
sugar	DECIMAL(6,2)	None	Simple sugars content measured strictly in grams (g).
sodium	DECIMAL(7,2)	None	Sodium mineral content measured strictly in milligrams (mg).
caffeine	DECIMAL(6,2)	None	Caffeine stimulant content measured strictly in milligrams (mg).
serving_size	VARCHAR(100)	None	Standard portion weight or volume (e.g. 350g, 400ml).
is_featured	TINYINT(1)	None	Boolean flag highlighting dish on restaurant entrance showcase.
is_popular	TINYINT(1)	None	Boolean flag designating popular best-seller badge.
is_available	TINYINT(1)	None	Live kitchen availability toggle (1=Available, 0=Sold Out).
created_at	TIMESTAMP	None	Food dish registration timestamp.
updated_at	TIMESTAMP	None	Food dish modification timestamp.

TABLE: FOOD_VARIANTS

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; food portion variant identifier.
food_item_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing food_items(id) ON DELETE CASCADE.

name	VARCHAR(100)	None	Portion title (e.g. Regular, Large Portion, Double Protein).
description	VARCHAR(255)	None	Explanation of portion size or preparation difference.
price_adjustment	DECIMAL(10,2)	None	Price differential added to base price in Indian Rupees (₹).
calories_adjustment	INT(11)	None	Calorie differential applied to base calories (kcal).
protein_adjustment	DECIMAL(6,2)	None	Protein differential applied to base protein (g).
carbs_adjustment	DECIMAL(6,2)	None	Carbohydrate differential applied to base carbs (g).
fat_adjustment	DECIMAL(6,2)	None	Fat differential applied to base fat (g).
fiber_adjustment	DECIMAL(6,2)	None	Fiber differential applied to base fiber (g).
sugar_adjustment	DECIMAL(6,2)	None	Sugar differential applied to base sugar (g).
sodium_adjustment	DECIMAL(7,2)	None	Sodium differential applied to base sodium (mg).
caffeine_adjustment	DECIMAL(6,2)	None	Caffeine differential applied to base caffeine (mg).
is_required	TINYINT(1)	None	Mandatory selection flag (default 0).
is_available	TINYINT(1)	None	Active availability toggle for the portion option.
sort_order	INT(11)	None	Display sort order within portion selection UI.
created_at	TIMESTAMP	None	Variant record creation timestamp.
updated_at	TIMESTAMP	None	Variant record modification timestamp.

TABLE: FOOD_CUSTOMIZATION

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; food customization add-on identifier.
food_item_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing food_items(id) ON DELETE CASCADE.
group_name	VARCHAR(100)	None	Option grouping header (e.g. Choose Base, Dressing, Extra Toppings).
name	VARCHAR(100)	None	Customization display title (e.g. Extra Hass Avocado, Greek Yogurt Dip).
description	VARCHAR(255)	None	Ingredient or preparation details of the add-on.
price_adjustment	DECIMAL(10,2)	None	Unit cost added to item total in Indian Rupees (₹).
calories_adjustment	INT(11)	None	Caloric energy added per unit selection (kcal).
protein_adjustment	DECIMAL(6,2)	None	Protein added per unit selection in grams (g).
carbs_adjustment	DECIMAL(6,2)	None	Carbohydrates added per unit selection in grams (g).
fat_adjustment	DECIMAL(6,2)	None	Fats added per unit selection in grams (g).
fiber_adjustment	DECIMAL(6,2)	None	Fiber added per unit selection in grams (g).
sugar_adjustment	DECIMAL(6,2)	None	Sugar added per unit selection in grams (g).
sodium_adjustment	DECIMAL(7,2)	None	Sodium added per unit selection in milligrams (mg).
caffeine_adjustment	DECIMAL(6,2)	None	Caffeine added per unit selection in milligrams (mg).

is_required	TINYINT(1)	None	Enforces at least one option selection in this group.
min_quantity	INT UNSIGNED	None	Minimum number of items customer must select from group (default 0).
max_quantity	INT UNSIGNED	None	Maximum number of items customer can select from group (default 1).
is_available	TINYINT(1)	None	Inventory availability toggle for the customization.
sort_order	INT(11)	None	Sort order within the customization group.
created_at	TIMESTAMP	None	Customization record creation timestamp.
updated_at	TIMESTAMP	None	Customization record modification timestamp.

TABLE: RESTAURANT_TABLE

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; physical dining table identifier.
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); parent restaurant entity.
branch_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing branches(id); physical branch location.
table_number	VARCHAR(50)	None	Display label identifying dining table (e.g. Table 1, Table 12).
status	ENUM('available','occupied','cleaning','out_of_service')	None	Live occupancy state (default 'available').
created_at	TIMESTAMP	None	Table configuration timestamp.
updated_at	TIMESTAMP	None	Table occupancy state transition timestamp.

TABLE: QR_TOKEN

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; cryptographic QR token identifier.
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); bound restaurant entity.
branch_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing branches(id); bound branch location.
table_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurant_tables(id); bound dining table.
token	VARCHAR(128)	UK	Cryptographically generated unique alphanumeric token string.
status	ENUM('active','inactive','expired')	None	Token security validation state (default 'active').
expires_at	TIMESTAMP	None	Optional expiry timestamp for time-limited QR sessions.
created_at	TIMESTAMP	None	Token generation and issuance timestamp.

TABLE: CUSTOMER

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; dining guest profile identifier.
name	VARCHAR(120)	None	Full name of the dining customer captured during checkout.
mobile	VARCHAR(30)	None	Customer contact mobile phone number.
email	VARCHAR(191)	None	Customer contact email address for electronic receipts.
created_at	TIMESTAMP	None	Customer profile creation timestamp.

updated_at	TIMESTAMP	None	Customer profile modification timestamp.
------------	-----------	------	--

TABLE: ORDER

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; master dining order identifier.
order_number	VARCHAR(64)	UK	Human-readable order tracking code (e.g. HB-1001, HB-18A3B4C-782).
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); target restaurant entity.
branch_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing branches(id); target dining outlet.
table_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurant_tables(id); NULL for Takeaway.
customer_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing customers(id); ordering guest diner.
order_type	ENUM('dine_in','takeaway')	None	Dining service mode (default 'dine_in').
subtotal	DECIMAL(10,2)	None	Sum of all order items and customizations before taxation (₹).
tax	DECIMAL(10,2)	None	Calculated Goods & Services Tax (GST 5%) in Indian Rupees (₹).
service_charge	DECIMAL(10,2)	None	Optional restaurant hospitality service charge in Indian Rupees (₹).
total_amount	DECIMAL(10,2)	None	Final grand payable total amount in Indian Rupees (₹).
payment_status	ENUM('pending','completed','failed')	None	Settlement state of order financial transaction.

order_status	ENUM('placed','accepted','preparing','ready','completed','cancelled')	None	Operational kitchen lifecycle status.
notes	TEXT	None	Customer special food preparation or dietary instructions.
created_at	TIMESTAMP	None	Order submission timestamp.
updated_at	TIMESTAMP	None	Order status transition timestamp.

TABLE: ORDER_ITEM

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; order line item identifier.
order_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing orders(id); master order parent.
food_item_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing food_items(id); source dish catalog item.
food_name_snapshot	VARCHAR(150)	None	Frozen dish name at the precise moment of purchase.
base_price_snapshot	DECIMAL(10,2)	None	Frozen baseline unit price at purchase time (₹).
variant_name_snapshot	VARCHAR(100)	None	Frozen portion variant title selected by customer.
variant_price_snapshot	DECIMAL(10,2)	None	Frozen portion price adjustment at purchase time (₹).
quantity	INT UNSIGNED	None	Quantity of this specific configured dish ordered.
unit_price	DECIMAL(10,2)	None	Calculated single-item price including variant & customizations (₹).
total_price	DECIMAL(10,2)	None	Line total = unit_price multiplied by quantity (₹).

calories	INT(11)	None	Frozen caloric energy snapshot per unit (kcal).
protein	DECIMAL(6,2)	None	Frozen protein snapshot per unit in grams (g).
carbs	DECIMAL(6,2)	None	Frozen carbohydrate snapshot per unit in grams (g).
fat	DECIMAL(6,2)	None	Frozen fat snapshot per unit in grams (g).
fiber	DECIMAL(6,2)	None	Frozen fiber snapshot per unit in grams (g).
sugar	DECIMAL(6,2)	None	Frozen sugar snapshot per unit in grams (g).
sodium	DECIMAL(7,2)	None	Frozen sodium snapshot per unit in milligrams (mg).
caffeine	DECIMAL(6,2)	None	Frozen caffeine snapshot per unit in milligrams (mg).
created_at	TIMESTAMP	None	Line item purchase timestamp.

TABLE: ORDER_ITEM_CUSTOMIZATION

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; order item customization snapshot identifier.
order_item_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing order_items(id) ON DELETE CASCADE.
customization_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing food_customizations(id).
customization_name_snapsho t	VARCHAR(100)	None	Frozen name of selected add-on option.
quantity	INT UNSIGNED	None	Quantity of this specific customization selected.
price_adjustment	DECIMAL(10,2)	None	Frozen price delta applied per unit add-on (₹).

calories_adjustment	INT(11)	None	Frozen calorie adjustment per unit (kcal).
protein_adjustment	DECIMAL(6,2)	None	Frozen protein adjustment per unit in grams (g).
carbs_adjustment	DECIMAL(6,2)	None	Frozen carbohydrate adjustment per unit in grams (g).
fat_adjustment	DECIMAL(6,2)	None	Frozen fat adjustment per unit in grams (g).
fiber_adjustment	DECIMAL(6,2)	None	Frozen fiber adjustment per unit in grams (g).
sugar_adjustment	DECIMAL(6,2)	None	Frozen sugar adjustment per unit in grams (g).
sodium_adjustment	DECIMAL(7,2)	None	Frozen sodium adjustment per unit in milligrams (mg).
caffeine_adjustment	DECIMAL(6,2)	None	Frozen caffeine adjustment per unit in milligrams (mg).
created_at	TIMESTAMP	None	Snapshot creation timestamp.

TABLE: PAYMENT

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; financial payment settlement identifier.
order_id	BIGINT(20) UNSIGNED	UK, FK	Foreign key referencing orders(id); strict 1-to-1 relationship.
payment_method	ENUM('cash','upi','card')	None	Authorized settlement mechanism.
transaction_reference	VARCHAR(128)	None	Unique transaction reference code (e.g. TXN-UPI-18A3B4C).
amount	DECIMAL(10,2)	None	Total settled financial amount in Indian Rupees (₹).
status	ENUM('pending','completed','failed')	None	Settlement transaction state (default 'pending').

paid_at	TIMESTAMP	None	Timestamp when payment was successfully verified and cleared.
created_at	TIMESTAMP	None	Payment record creation timestamp.

TABLE: REVIEW

FIELD NAME	DATA TYPE	KEY	DESCRIPTION
id	BIGINT(20) UNSIGNED	PK	Primary key; customer feedback review identifier.
restaurant_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing restaurants(id); target restaurant entity.
customer_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing customers(id); authoring guest customer.
order_id	BIGINT(20) UNSIGNED	FK	Foreign key referencing orders(id); specific dining order reviewed.
rating	TINYINT UNSIGNED	None	Numerical dining quality rating on 1 to 5 star scale.
comment	TEXT	None	Qualitative feedback text submitted by the customer.
restaurant_reply	TEXT	None	Official restaurant owner response reply published to customer.
replied_at	TIMESTAMP	None	Timestamp when restaurant owner published the response reply.
status	ENUM('pending','approved','hidden')	None	Public review moderation state (default 'pending').
created_at	TIMESTAMP	None	Review submission timestamp.
updated_at	TIMESTAMP	None	Review or reply modification timestamp.

7.IMPLEMENTATION

Figure 7.1: Customer Welcome & Table Selection Portal

Initial landing screen allowing walk-in diners to select dining options or initiate table sessions.

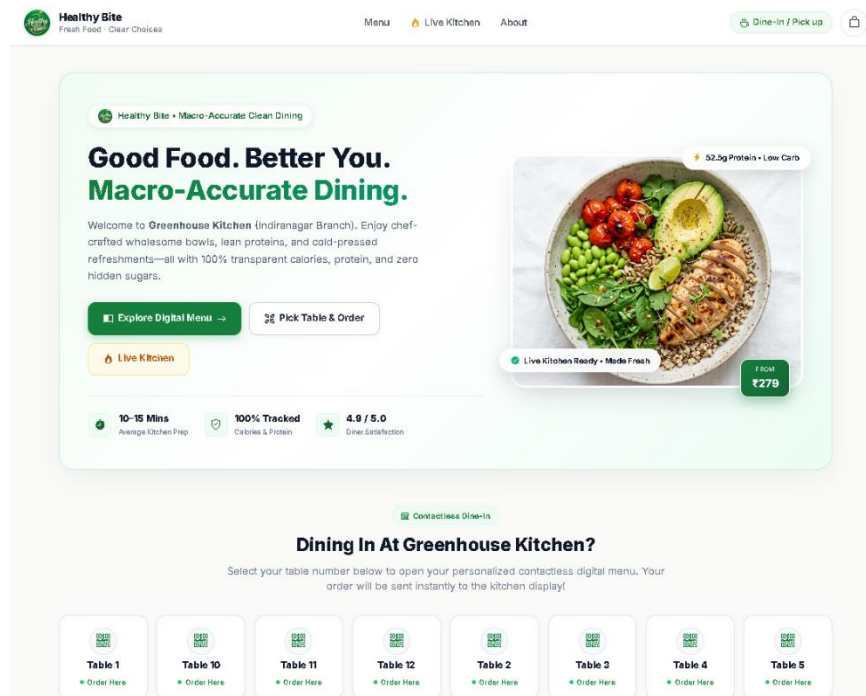


Figure 7.2: Interactive Customer Digital Menu with Category Filters

Categorized gourmet food catalog featuring real Unsplash photography, diet lifestyle badges, and calorie indicators.

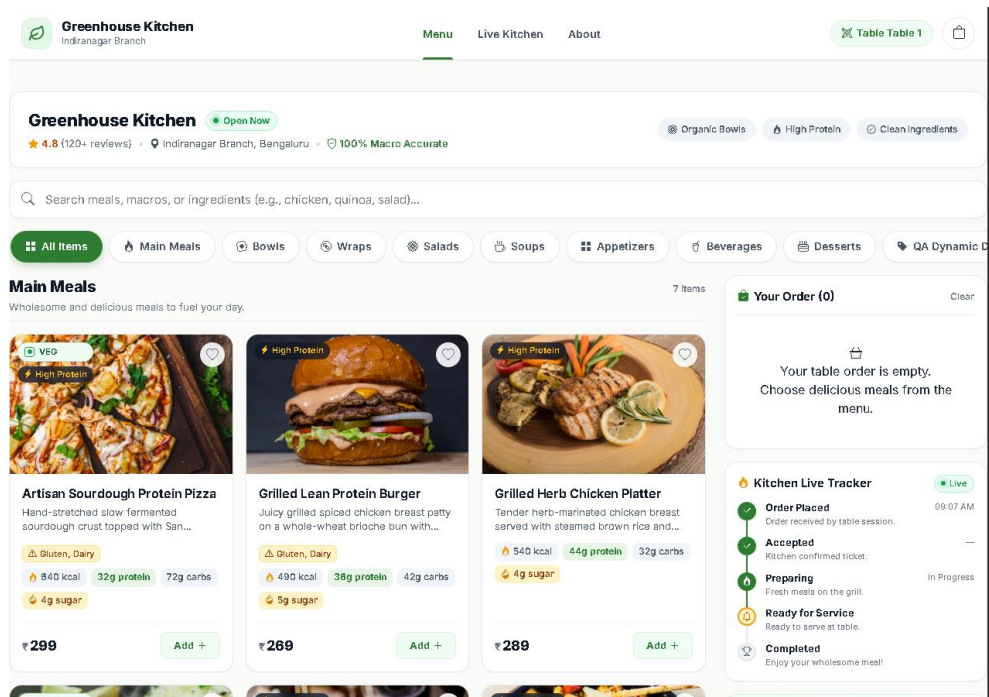


Figure 7.3: Customer Order Checkout & Table Context Verification
Checkout screen displaying bound dining Table 4, order summary, guest contact form, and payment method selectors.

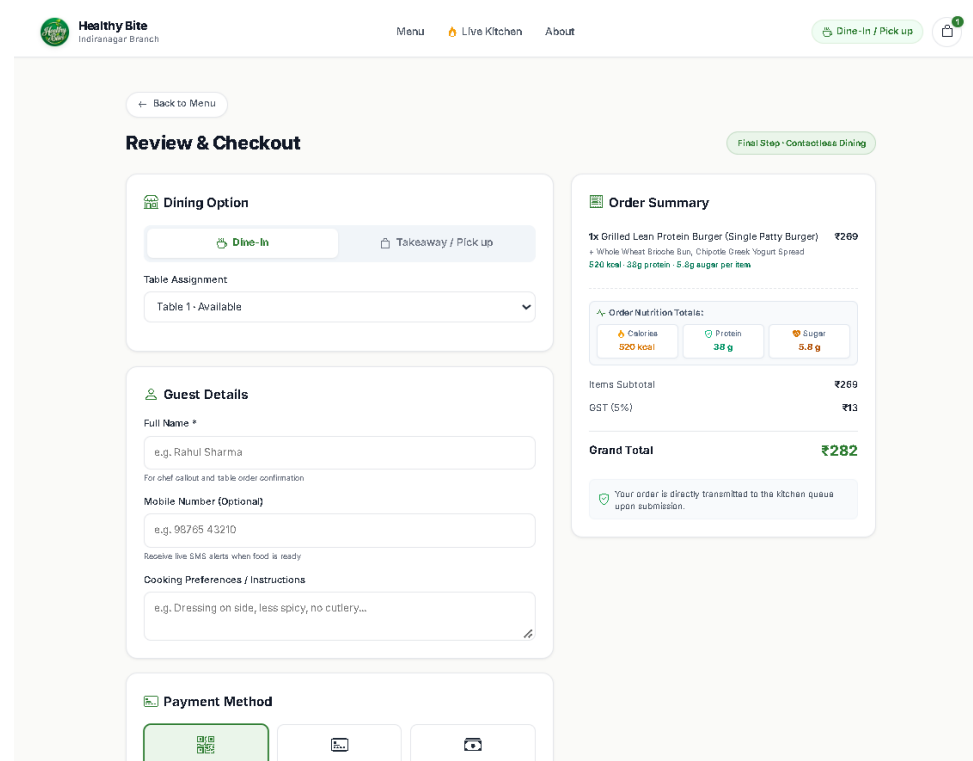


Figure 7.4: Customer Order Confirmation & Electronic Receipt
Post-order confirmation displaying unique tracking code (HB-1001), item breakdown, subtotal, and tax invoice details.

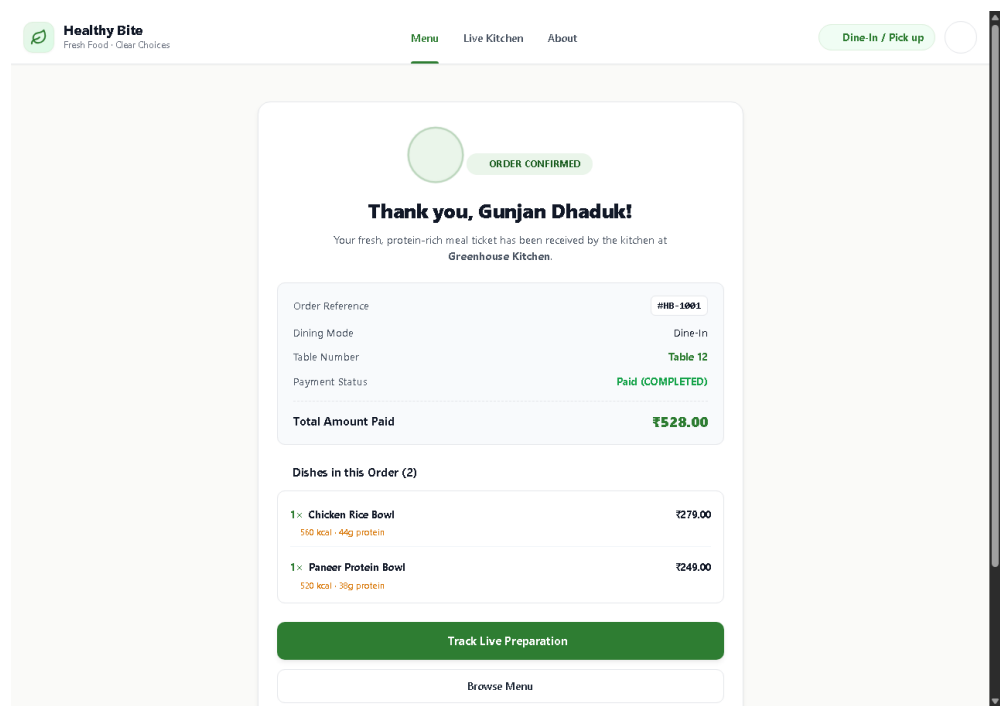


Figure 7.5: Live Customer Kitchen Order Tracking Screen

Real-time 5-stage visual progress timeline polled asynchronously from the server without page reloads.

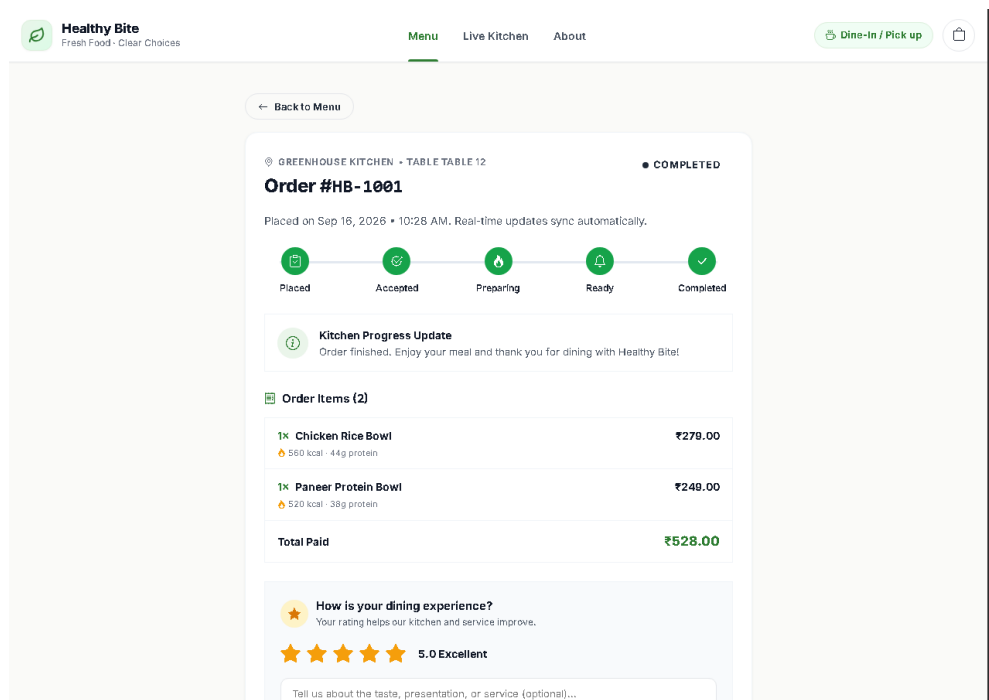


Figure 7.6: Restaurant Partner Portal Sign In Screen

Secure authentication interface for restaurant owners, managers, and kitchen staff with email normalization and BCrypt verification.

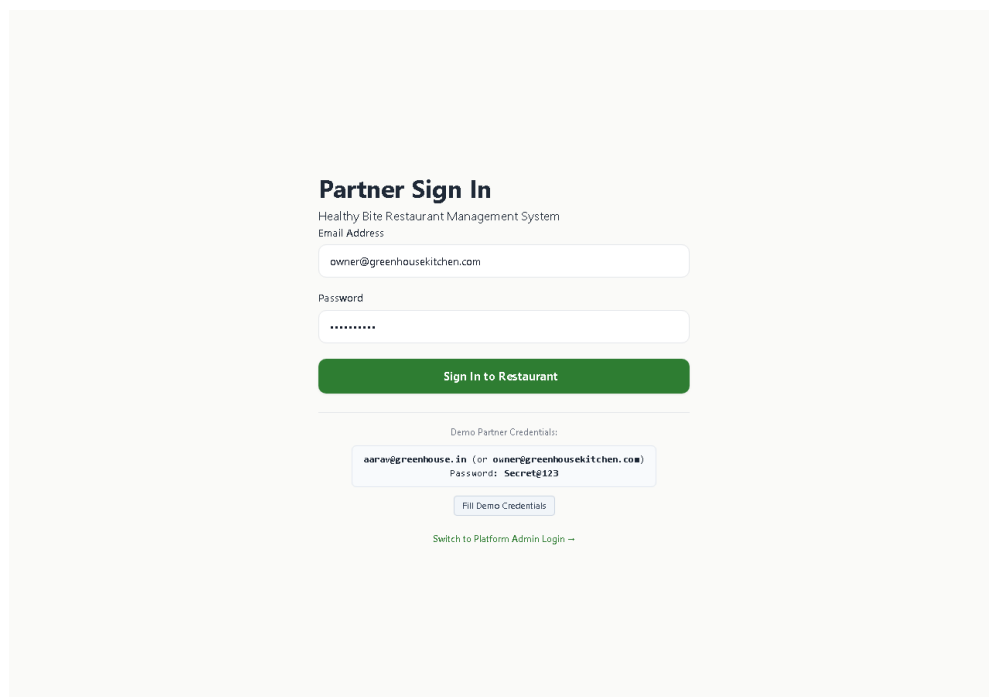


Figure 7.7: Restaurant Owner Overview Monitoring Dashboard

Central operational command center displaying 6 core KPI stat cards, sales trend bezier curves, order completion donut charts, and recent orders.

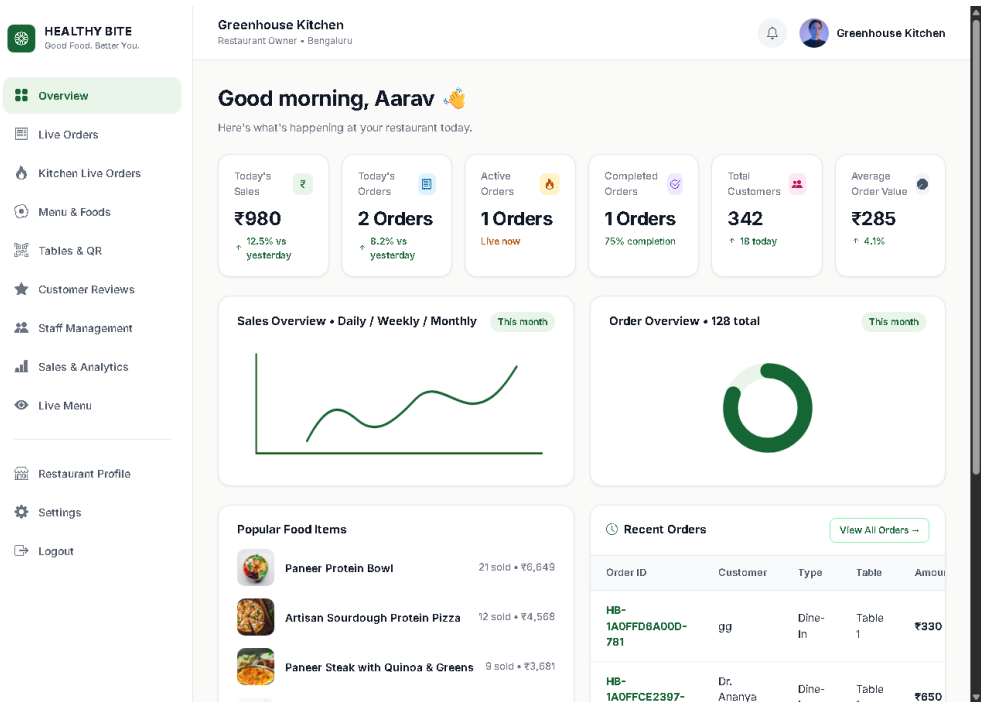


Figure 7.8: Live Orders Monitoring View with Filter Legend Pills

Order monitoring view featuring real-time status count pills (Placed, Accepted, Preparing, Ready, Completed, Cancelled) and itemized receipt modal.

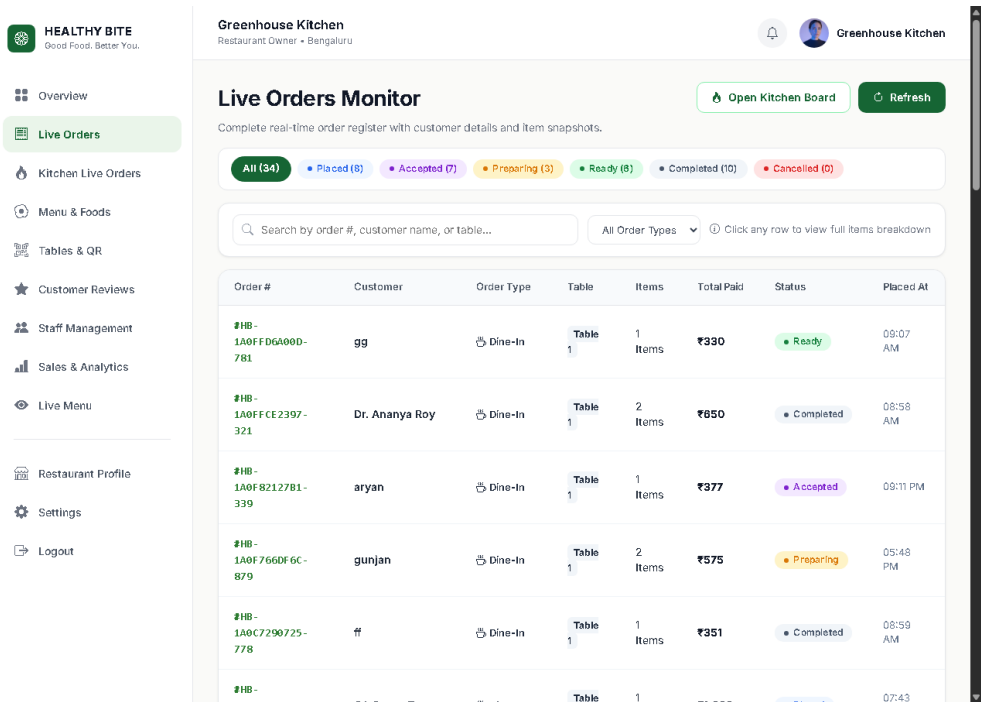


Figure 7.9: 5-Column Operational Kitchen Live Orders Kanban Board

Dedicated kitchen fulfillment board displaying ticket cards with customization snapshots and operational stage-advancement buttons.

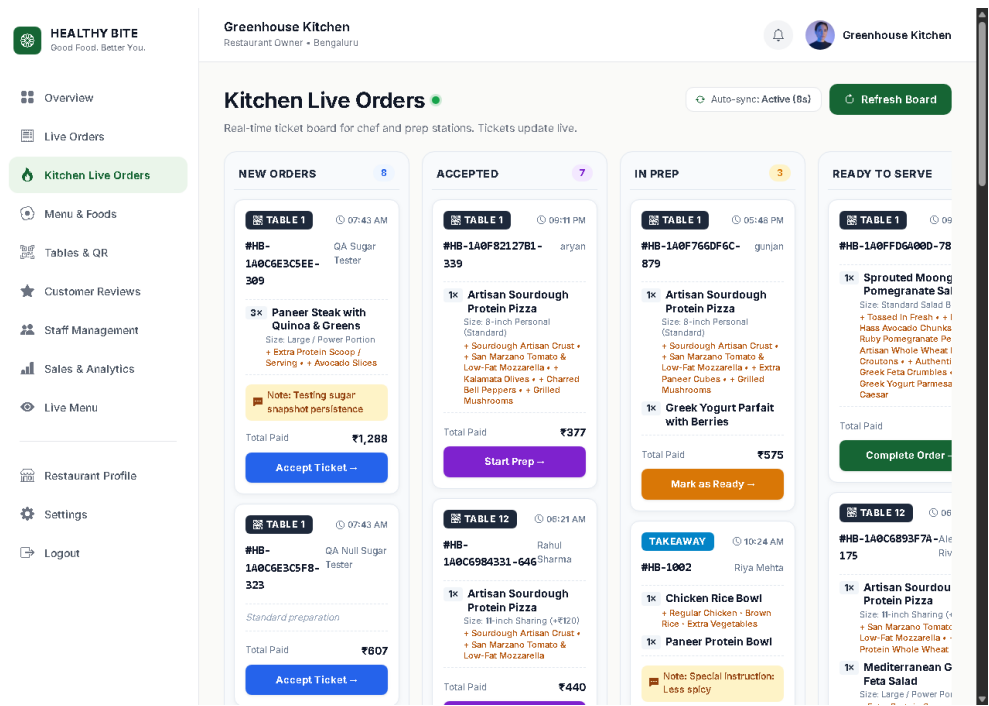


Figure 7.10: Menu & Foods Catalog CRUD Management

Comprehensive food dishes table displaying dish photos, categories, base prices, calories, protein, and active availability toggle switches.

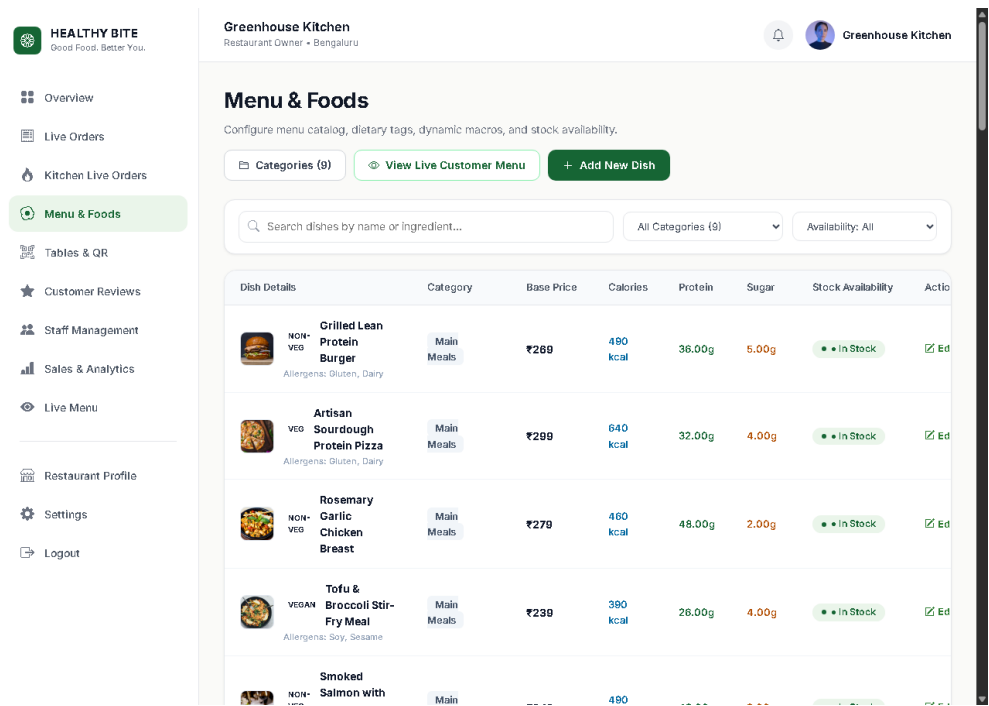
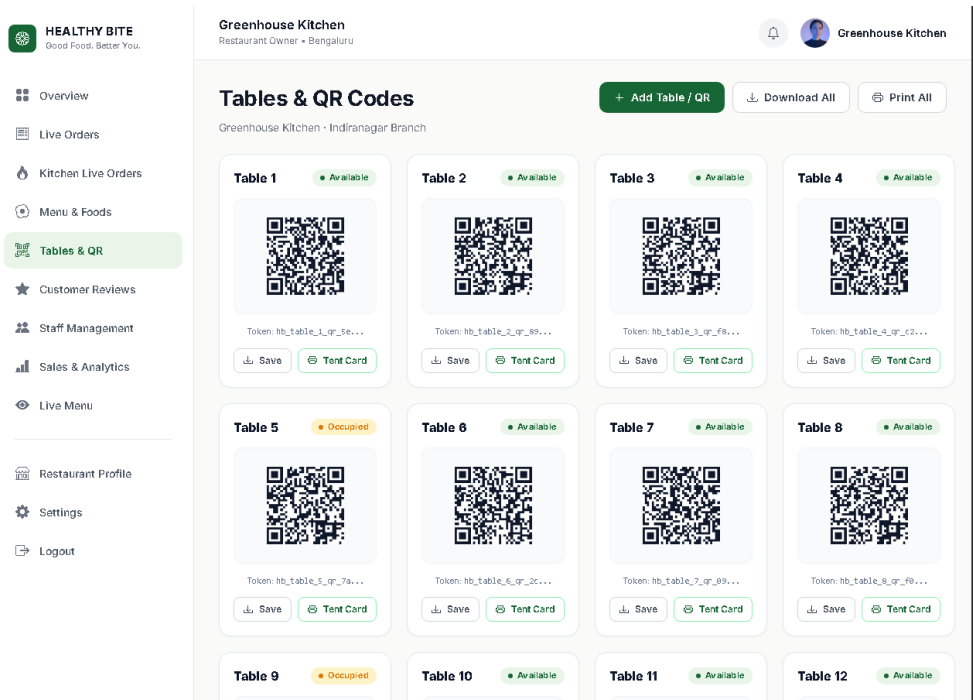


Figure 7.11: Tables 1–12 Management & Scannable QR Code Generators

Dining room table grid displaying canvas-rendered QR codes, 1-click PNG download buttons, print ticket popups, and click-to-toggle occupancy pills.

**Figure 7.12: Customer Reviews Management with Reply Publishing**

Customer dining feedback management screen featuring star rating filters, quick-reply suggestion chips, and persistent response publishing.

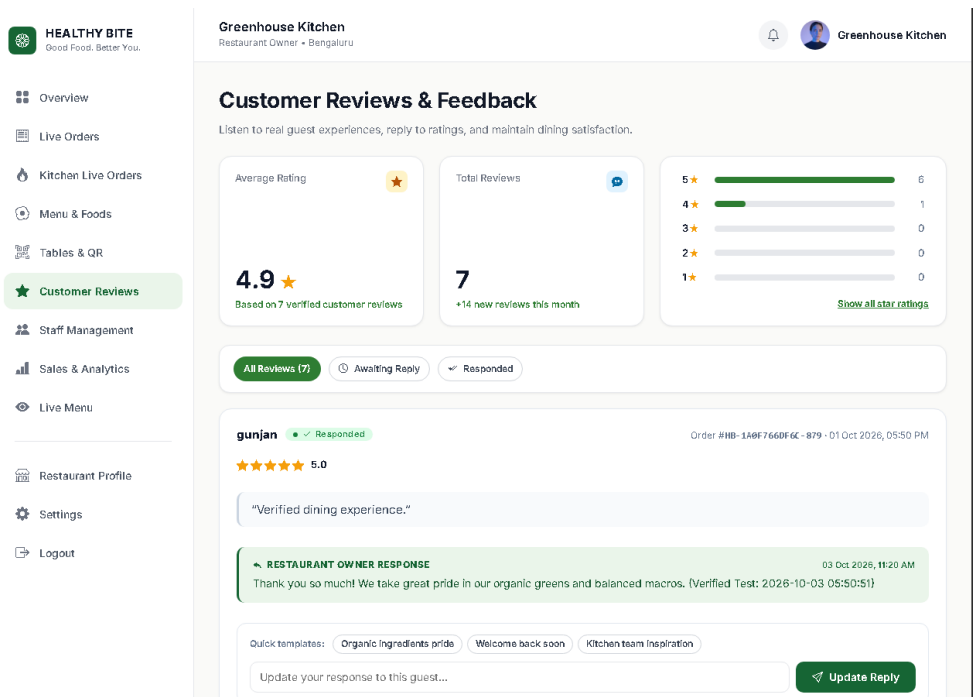


Figure 7.13: Restaurant Staff & Team Access Management

Staff access directory displaying team members, assigned authorization roles (Manager, Staff, Waiter), and active/inactive status switches.

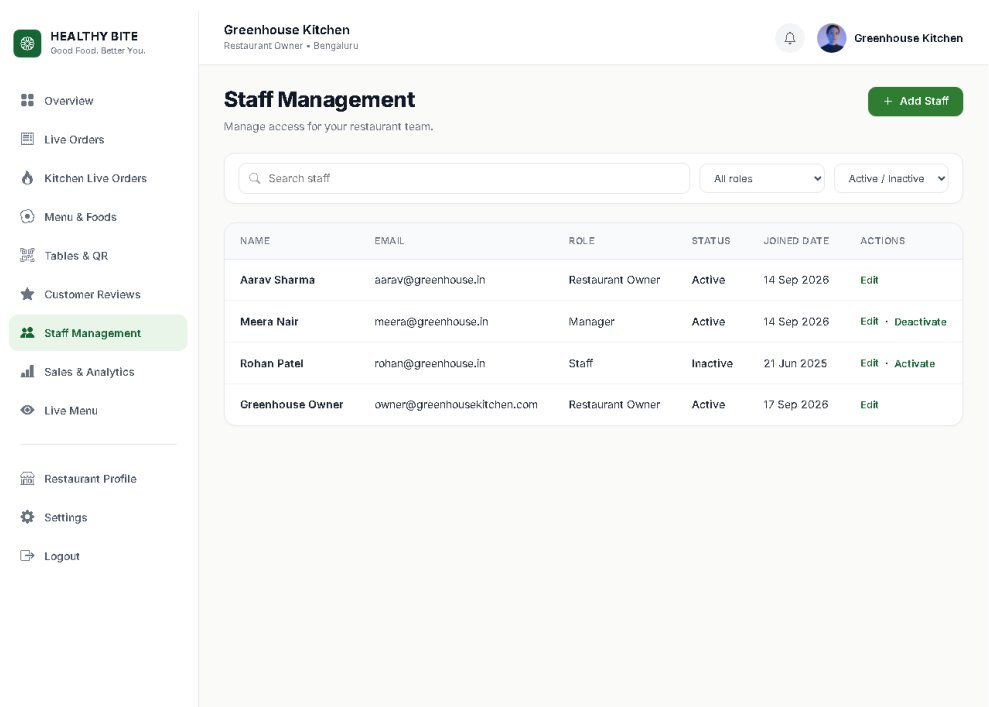


Figure 7.14: Sales & Macro Nutrient Analytics Dashboard

Financial and dietary analytics displaying revenue trends, nutrient macro percentages (Protein, Carbs, Fat), and payment method distribution.

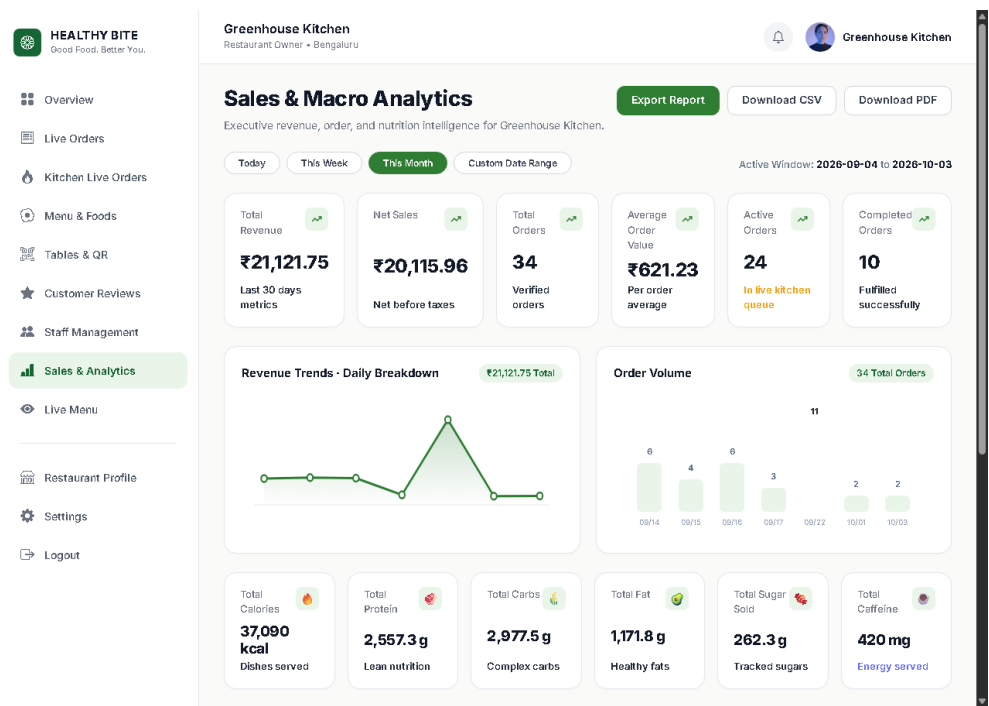


Figure 7.15: Live Customer Digital Menu Preview Simulation

High-fidelity live menu simulation preview displaying high-resolution food photos, calorie badges, and interactive cart drawer.

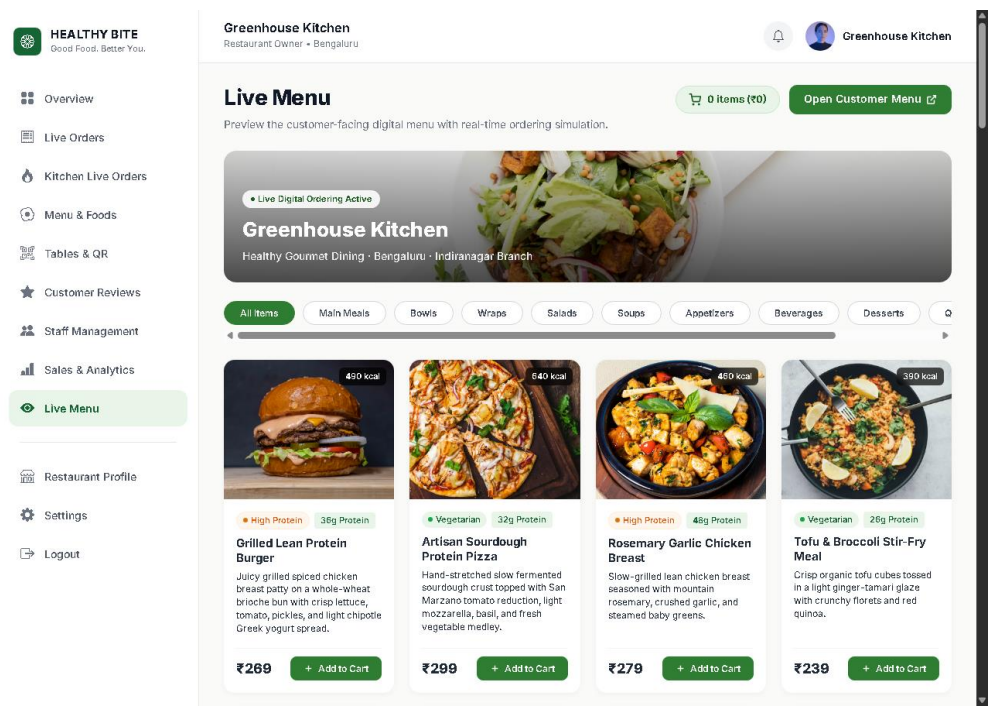


Figure 7.16: Restaurant Branding & Profile Settings

Restaurant administrative profile editor managing trade name, contact telephone, email, city, state, and registered physical address.

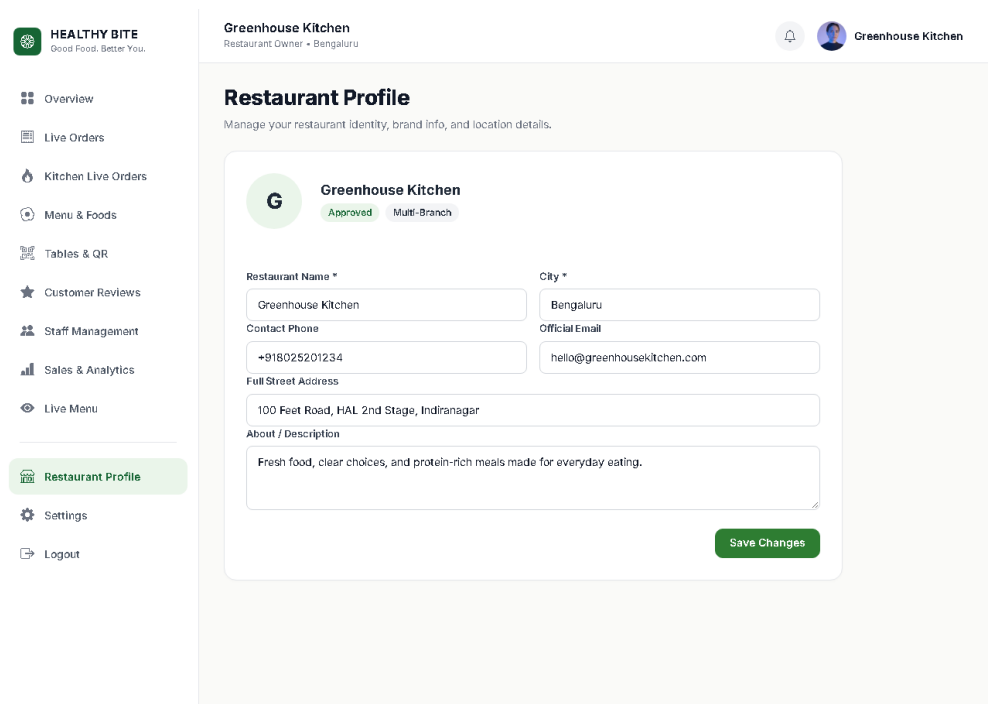


Figure 7.17: Operational Preferences & System Settings

Restaurant operational preferences managing automatic order acceptance, table ordering toggles, tax rates (5% GST), and service charges.

The screenshot shows the 'Settings' page for 'Greenhouse Kitchen', a restaurant owner in Bengaluru. The left sidebar contains navigation links: Overview, Live Orders, Kitchen Live Orders, Menu & Foods, Tables & QR, Customer Reviews, Staff Management, Sales & Analytics, Live Menu, Restaurant Profile, **Settings** (highlighted), and Logout. The main content area is titled 'Settings' and includes a subtitle: 'Configure notification preferences, operational alerts, and account security.' It features two sections: 'Operational Preferences' and 'Account Security'. The 'Operational Preferences' section has three items: 'Automatic Kitchen Sound Alerts' (checked), 'Live Order Auto-Accept' (unchecked), and 'Customer Nutrition Breakdown Display' (checked). The 'Account Security' section includes fields for 'Current Password', 'New Password', and 'Confirm New Password', with a 'Minimum 6 characters' hint. A green 'Update Security' button is at the bottom right.

Figure 7.18: Platform Super Admin Sign In Portal

Dedicated platform governance entrance for system administrators.

The screenshot shows the 'Platform Admin' sign-in portal. It features a title 'Platform Admin' and a subtitle 'Healthy Bite System Infrastructure & Multi-Tenancy'. Below this are fields for 'Admin Email' (with the example 'admin@healthybite.com') and 'Master Password'. A blue 'Sign In to Admin Console' button is positioned below the password field. Underneath, a section for 'Root Credentials' displays 'mir@healthybite.in (or: admin@healthybite.com)' and 'Password: Secret@123'. A 'Fill Demo Credentials' button is located below the root credentials. At the bottom, there is a link to 'Switch to Restaurant Owner Portal'.

Figure 7.19: Platform Overview & Multi-Tenant Governance Dashboard

Cross-tenant executive overview displaying 6 platform KPI stat cards, 4 analytical SVG charts, and recent restaurant registrations.

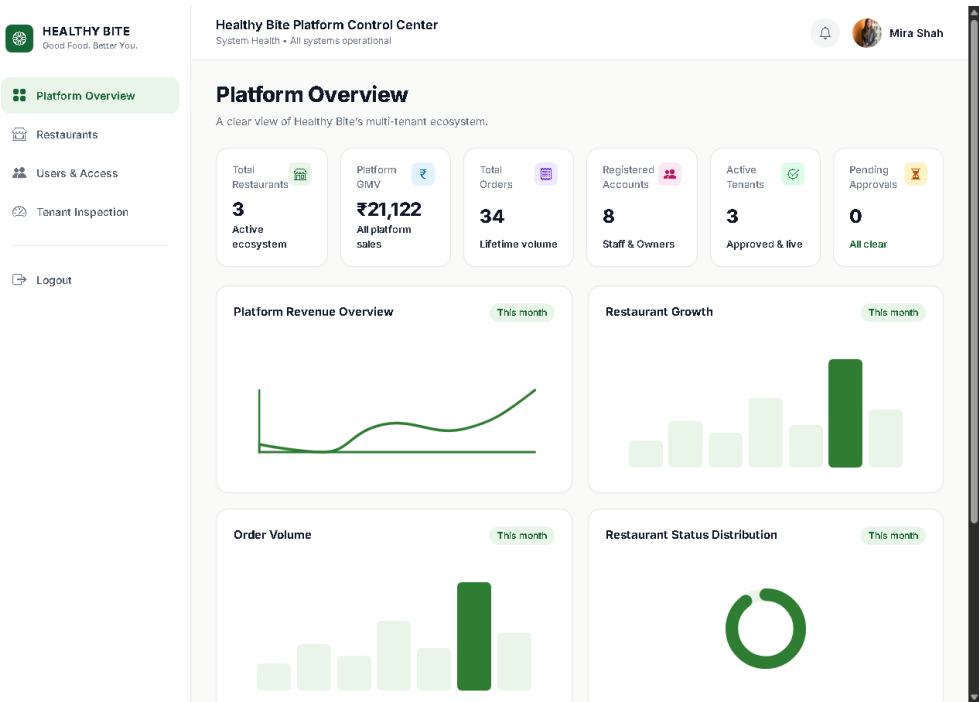


Figure 7.20: Registered Restaurants Directory with Status Filter Cards

Multi-tenant restaurant directory featuring interactive status filter cards (Approved, Pending, Suspended) and inline status dropdowns.

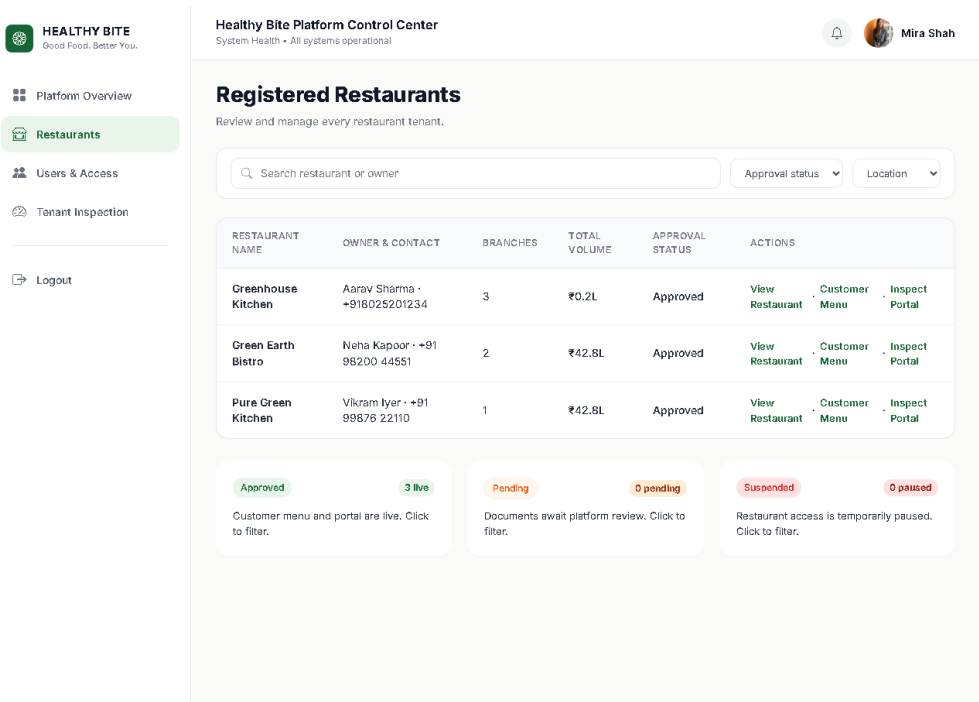


Figure 7.21: Deep Restaurant Tenant Portal Inspection Audit

In-depth tenant audit inspection screen displaying branch summaries, menu dish counts, dining tables, and monthly revenue performance.

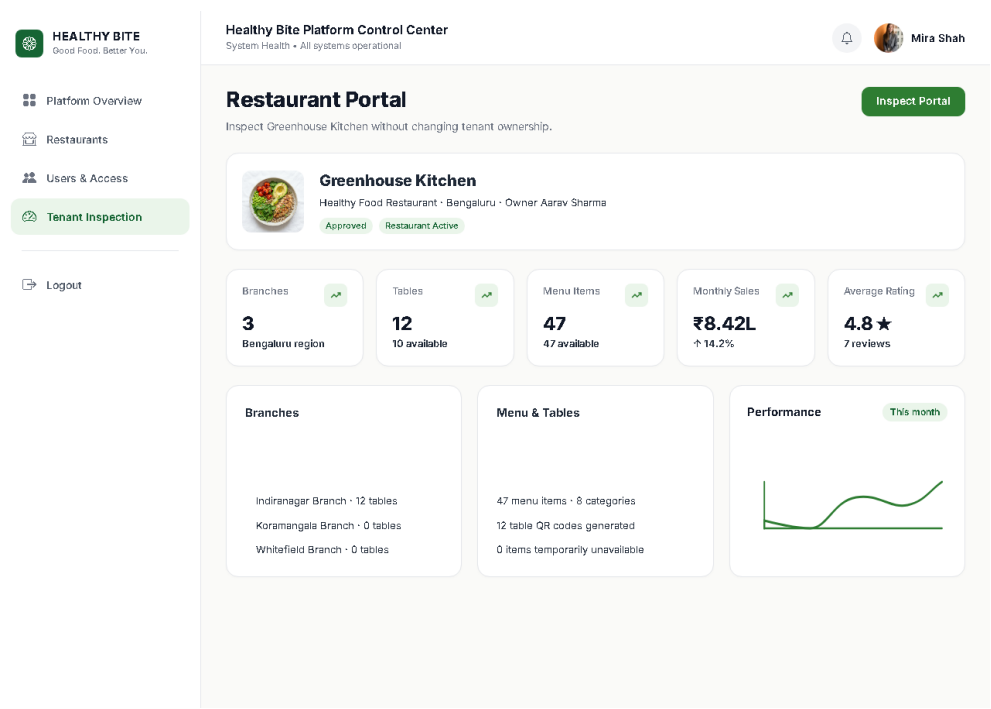
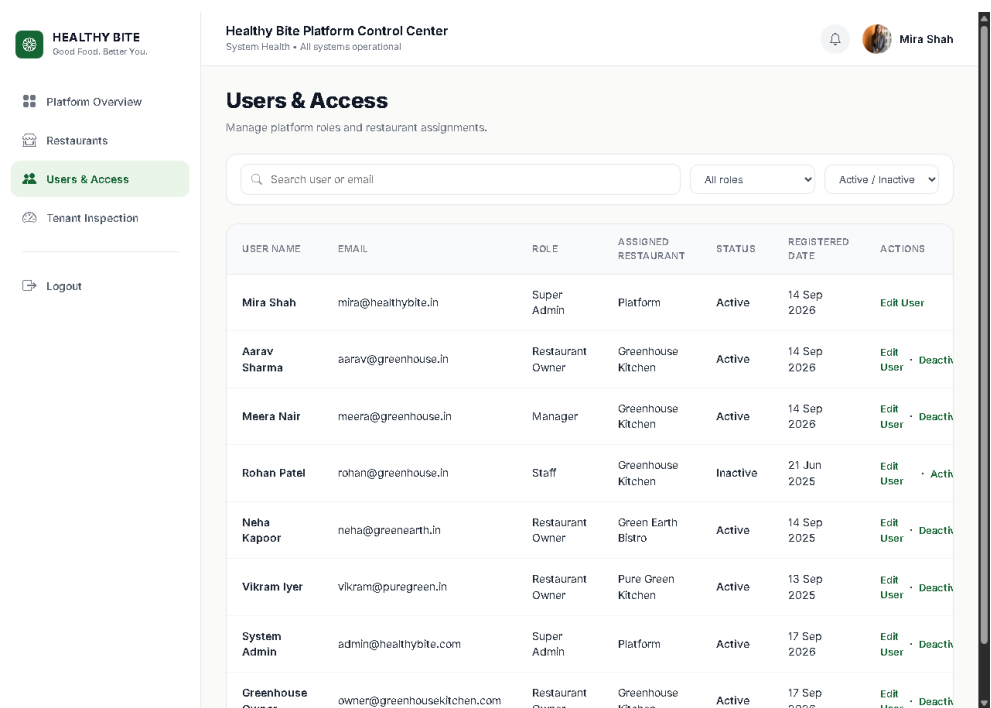


Figure 7.22: Platform Users & Role Authority Management

Platform-wide user management directory displaying user accounts, role assignments, tenant scoping, and activation toggles.



8.CODE EXPLANATION

Rather than including exhaustive project listings, this chapter isolates and examines 18 crucial architectural algorithms and production code snippets extracted directly from the verified Healthy Bite codebase. Each section provides the exact source file path, architectural module, primary purpose, line reference, formatted code snippet, in-depth technical explanation, and verified expected behavior.

Source File: `public/index.php` | *Module: Architecture & Front Controller* | *Reference: Lines 1–52*

Purpose: Application entry point, PSR-4 fallback autoloader, helper loading, and lifecycle dispatch

```
<?php
declare(strict_types=1);

define('HB_ROOT', dirname(__DIR__));

// 1. PSR-4 Fallback Autoloader
spl_autoload_register(function (string $class) {
    $prefix = 'App\\';
    $baseDir = HB_ROOT . '/app/';
    $len = strlen($prefix);
    if (strncmp($prefix, $class, $len) !== 0) return;
    $relativeClass = substr($class, $len);
    $file = $baseDir . str_replace('\\', '/', $relativeClass) . '.php';
    if (file_exists($file)) require_once $file;
});

// 2. Load Helpers & Constants
require_once HB_ROOT . '/config/constants.php';
require_once HB_ROOT . '/app/Helpers/url.php';
require_once HB_ROOT . '/app/Helpers/format.php';
require_once HB_ROOT . '/app/Helpers/security.php';
require_once HB_ROOT . '/app/Helpers/food.php';

// 3. Load Environment & Dispatch
\\App\\Core\\Env::load(HB_ROOT . '/.env');
$app = new \\App\\Core\\App();
$router = $app->getRouter();
require_once HB_ROOT . '/routes/web.php';
require_once HB_ROOT . '/routes/api.php';
$app->run();
```

Technical Explanation: Acts as the single gatekeeper for all HTTP requests. Automatically handles static file serving in CLI server mode, registers the custom PSR-4 autoloader, initializes environment variables, registers web and API routes, and triggers the request dispatcher.

Expected System Behavior: Every incoming browser or API request is intercepted, routed through security filters and middleware, and cleanly dispatched to the appropriate controller action.

Source File: `app/Core/Router.php` | *Module: Routing Engine* | *Reference: Lines 35–82*

Purpose: Dynamic regex pattern route matching with HTTP method enforcement and parameter extraction

```
public function dispatch(Request $request): Response
{
    $method = $request->getMethod();
    $path = $request->getPath();

    foreach ($this->routes as $route) {
        if ($route['method'] !== $method) continue;

        $pattern = preg_replace('/^([a-zA-Z0-9_+])\./', '{P<$1>[^\./]+}', $route['uri']);
        $pattern = '#^' . $pattern . '$#';

        if (preg_match($pattern, $path, $matches)) {
            $params = array_filter($matches, 'is_string', ARRAY_FILTER_USE_KEY);
```

```
        return $this->executeHandler($route['handler'], $params);
    }
}
return Response::html('<h1>404 Not Found</h1>', 404);
}
```

Technical Explanation: Iterates through registered routes, converts dynamic URI segments (such as {orderId} or {id}) into named regular expression capture groups, extracts parameters, and invokes controller handlers.

Expected System Behavior: Accurately resolves dynamic URLs like /menu/confirmation/HB-1001 and passes extracted parameters directly to controller action arguments.

Source File: `app/Core/Database.php` | *Module: Data Abstraction Layer* | *Reference: Lines 18–45*

Purpose: Singleton PDO connection manager with secure configuration and prepared statement guarantees

```
public static function getConnection(): PDO
{
    if (self::$instance === null) {
        $config = require HB_ROOT . '/config/database.php';
        $dsn = sprintf('mysql:host=%s;port=%s;dbname=%s;charset=%s',
            $config['host'], $config['port'], $config['database'], $config['charset']
        );
        self::$instance = new PDO($dsn, $config['username'], $config['password'], [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,
        ]);
    }
    return self::$instance;
}
```

Technical Explanation: Implements the Singleton design pattern to maintain a single reusable PDO connection to MySQL. Strictly disables emulated prepared statements to ensure all SQL queries are parsed and executed securely on the MySQL server.

Expected System Behavior: Protects against SQL injection vulnerabilities across all repository queries while minimizing database connection overhead.

Source File: `app/Core/Auth.php` | *Module: Security & Identity* | *Reference: Lines 10–44*

Purpose: Role verification, session authentication state, and credential encapsulation

```
public static function check(): bool
{
    Session::start();
    return Session::has(self::USER_SESSION_KEY);
}

public static function user(): ?array
{
    Session::start();
    return Session::get(self::USER_SESSION_KEY);
}

public static function login(array $user): void
{
    Session::start();
    Session::regenerate();
    Session::set(self::USER_SESSION_KEY, [
        'id' => $user['id'],
        'role_id' => $user['role_id'],
        'restaurant_id' => $user['restaurant_id'] ?? null,
        'name' => $user['name'],
        'email' => $user['email'],
    ]);
}
```

Technical Explanation: Provides a clean interface for session identity. On login, automatically regenerates the session ID to thwart session fixation attacks and encapsulates user ID, role ID, and restaurant scoping parameters.

Expected System Behavior: Enforces strict authorization boundaries, preventing unauthorized access across Super Admin, Restaurant Owner, Staff, and Guest portals.

Source File: `app/Services/QrService.php` | *Module: Contactless Dining* | *Reference: Lines 22–58*

Purpose: Cryptographic QR token resolution and multi-entity relational table context binding

```
public function resolveToken(string $token): ?array
{
    $sql = "SELECT qr.id AS qr_token_id, qr.token, qr.status AS qr_status,
        t.id AS table_id, t.table_number, t.status AS table_status,
        b.id AS branch_id, b.name AS branch_name,
        r.id AS restaurant_id, r.name AS restaurant_name
    FROM qr_tokens qr
    JOIN restaurant_tables t ON qr.table_id = t.id
    JOIN branches b ON t.branch_id = b.id
    JOIN restaurants r ON b.restaurant_id = r.id
    WHERE qr.token = :token AND qr.status = 'active'
        AND r.status = 'approved' AND b.status = 'active'
    LIMIT 1";

    $stmt = $this->db->prepare($sql);
    $stmt->execute([':token' => $token]);
    return $stmt->fetch() ?: null;
}
```

Technical Explanation: Executes an inner-join query across `qr_tokens`, `restaurant_tables`, `branches`, and `restaurants`. Validates that the token is active, the branch is operational, and the restaurant is approved by the Super Admin.

Expected System Behavior: Binds customer browsing sessions directly to the physical dining table and prevents spoofing of table numbers or restaurant identities.

Source File: `app/Repositories/FoodRepository.php` | *Module: Food Catalog Management* | *Reference: Lines 28–64*

Purpose: Restaurant-scoped food dish retrieval with full 8-macro nutritional metrics and diet tags

```
public function getByRestaurantId(int $restaurantId, ?int $categoryId = null, ?string $search = null): array
{
    $sql = "SELECT f.*, c.name AS category_name
    FROM food_items f
    JOIN categories c ON f.category_id = c.id
    WHERE f.restaurant_id = :restaurant_id AND f.is_available = 1";
    $params = [':restaurant_id' => $restaurantId];

    if ($categoryId !== null) {
        $sql .= " AND f.category_id = :category_id";
        $params[':category_id'] = $categoryId;
    }

    if ($search !== null && $search !== "") {
        $sql .= " AND (f.name LIKE :search OR f.description LIKE :search)";
        $params[':search'] = '%' . $search . '%';
    }

    $sql .= " ORDER BY c.sort_order ASC, f.id ASC";
    $stmt = $this->db->prepare($sql);
    $stmt->execute($params);
    return $stmt->fetchAll();
}
```

Technical Explanation: Fetches dishes strictly belonging to the authorized restaurant. Enforces category filtering, search term matching, and active availability checks.

Expected System Behavior: Ensures complete multi-tenant catalog isolation so customers only view dishes belonging to the scanned restaurant.

Source File: `app/Services/PricingService.php` | *Module: Pricing Engine* | *Reference: Lines 18–55*

Purpose: Mathematical calculation of line item prices, variants, customization sums, and 5% GST

```
public function calculateSingleItemPrice(float $basePrice, float $variantAdj, array $customizations): float
{
    $customTotal = 0.0;
    foreach ($customizations as $c) {
        $qty = max(1, (int)($c['quantity'] ?? 1));
        $customTotal += (float)$c['price_adjustment'] * $qty;
    }
    return round($basePrice + $variantAdj + $customTotal, 2);
}

public function calculateOrderTotals(array $lineTotals, float $taxRate = 0.05, float $serviceCharge = 0.0): array
{
    $subtotal = round(array_sum($lineTotals), 2);
    $tax = round($subtotal * $taxRate, 2);
    $totalAmount = round($subtotal + $tax + $serviceCharge, 2);
    return [
        'subtotal' => $subtotal,
        'tax' => $tax,
        'service_charge' => $serviceCharge,
        'total_amount' => $totalAmount
    ];
}
```

Technical Explanation: Calculates unit prices by summing base price, portion variant price adjustment, and all selected customization add-ons. Applies standard 5% GST tax and produces verifiable grand totals.

Expected System Behavior: Guarantees mathematical precision across cart drawer, checkout calculations, and database order snapshots.

Source File: `app/Services/NutritionService.php` | *Module: Nutrition Calculation Engine* | *Reference: Lines 25–85*

Purpose: Comprehensive arithmetic summation and quantity scaling across all 8 nutrient macros

```
public function calculateItemNutrition(array $baseItem, ?array $variant, array $customizations): array
{
    $macroKeys = ['calories', 'protein', 'carbs', 'fat', 'fiber', 'sugar', 'sodium', 'caffeine'];
    $result = [];

    foreach ($macroKeys as $m) {
        $base = $baseItem[$m] ?? null;
        if ($base === null) {
            $result[$m] = null;
            continue;
        }
        $val = (float)$base;
        if ($variant && isset($variant[$m . '_adjustment']) && $variant[$m . '_adjustment'] !== null) {
            $val += (float)$variant[$m . '_adjustment'];
        }
        foreach ($customizations as $c) {
            if (isset($c[$m . '_adjustment']) && $c[$m . '_adjustment'] !== null) {
                $qty = max(1, (int)($c['quantity'] ?? 1));
                $val += (float)$c[$m . '_adjustment'] * $qty;
            }
        }
        $result[$m] = ($m === 'calories') ? (int)round(max(0, $val)) : round(max(0.0, $val), 2);
    }
    return $result;
}
```

Technical Explanation: Evaluates each of the 8 nutrient macros individually. Preserves NULL values if unconfigured, adds portion deltas and customization deltas, and scales integers for calories and decimals for macros.

Expected System Behavior: Displays scientifically accurate nutritional breakdowns for health-conscious diners and records immutable macro snapshots on purchase.

Source File: `app/Services/CartService.php` | *Module: Cart Verification* | *Reference: Lines 30–88*

Purpose: Server-side re-validation of client cart items against active MySQL records to prevent price tampering

```
public function validateCartItems(array $rawCartItems, int $restaurantId): array
{
    $validatedItems = [];
    foreach ($rawCartItems as $raw) {
        $foodId = (int)($raw['food_id'] ?? 0);
        $food = $this->foodRepo->getByIdAndRestaurant($foodId, $restaurantId);
        if (!$food || !(bool)$food['is_available']) continue;

        $variant = null;
        if (!empty($raw['variant_id'])) {
            $variant = $this->foodRepo->getVariantById((int)$raw['variant_id'], $foodId);
        }

        $customizations = [];
        if (!empty($raw['customization_ids'])) {
            $customizations = $this->foodRepo->getCustomizationsByIds((array)$raw['customization_ids'], $foodId);
        }

        $qty = max(1, (int)($raw['quantity'] ?? 1));
        $unitPrice = $this->pricingService->calculateSingleItemPrice((float)$food['base_price'], (float)($variant['price_adjustment'] ?? 0), $customizations);
        $nutrition = $this->nutritionService->calculateItemNutrition($food, $variant, $customizations);

        $validatedItems[] = [
            'food'      => $food,
            'variant'    => $variant,
            'customizations' => $customizations,
            'quantity'   => $qty,
            'unit_price' => $unitPrice,
            'line_total' => round($unitPrice * $qty, 2),
            'nutrition'  => $nutrition
        ];
    }
    return $validatedItems;
}
```

Technical Explanation: Intercepts client-submitted cart arrays during checkout. Re-fetches prices, variants, and customizations directly from MySQL, completely ignoring any client-modified price fields.

Expected System Behavior: Protects restaurant operators from client-side payload tampering and malicious price manipulations.

Source File: `app/Services/OrderService.php` | *Module: Order Fulfillment Pipeline* | *Reference: Lines 40–115*

Purpose: End-to-end transactional order creation, token locking, customer resolution, and snapshot persistence

```
public function placeOrder(array $payload): array
{
    $db = Database::getConnection();
    $db->beginTransaction();
    try {
        // Enforce verified QR token context over client payload
        if (!empty($payload['qr_token'])) {
            $resolved = $this->qrService->resolveToken($payload['qr_token']);
            if ($resolved) {
                $payload['restaurant_id'] = $resolved['restaurant_id'];
                $payload['branch_id']     = $resolved['branch_id'];
                $payload['table_id']      = $resolved['table_id'];
            }
        }

        $validatedItems = $this->cartService->validateCartItems($payload['items'], (int)$payload['restaurant_id']);
        if (empty($validatedItems)) throw new \Exception('Cart contains no valid items.');
```

```
        $customerId = $this->orderRepo->findOrCreateCustomer($payload['customer_name'], $payload['customer_mobile'],
```

```

$payload['customer_email'] ?? null);
$orderNumber = 'HB-' . strtoupper(dechex(time())) . '-' . rand(100, 999);

$lineTotals = array_column($validatedItems, 'line_total');
$totals = $this->pricingService->calculateOrderTotals($lineTotals);

$orderId = $this->orderRepo->createOrder([
    'order_number' => $orderNumber,
    'restaurant_id' => $payload['restaurant_id'],
    'branch_id' => $payload['branch_id'],
    'table_id' => $payload['table_id'] ?? null,
    'customer_id' => $customerId,
    'order_type' => $payload['order_type'] ?? 'dine_in',
    'subtotal' => $totals['subtotal'],
    'tax' => $totals['tax'],
    'total_amount' => $totals['total_amount'],
    'order_status' => 'placed',
    'payment_status' => 'pending'
]);

foreach ($validatedItems as $item) {
    $orderItemId = $this->orderRepo->addOrderItem($orderId, $item);
    foreach ($item['customizations'] as $c) {
        $this->orderRepo->addOrderItemCustomization($orderItemId, $c);
    }
}

$db->commit();
return ['success' => true, 'order_id' => $orderId, 'order_number' => $orderNumber];
} catch (\Throwable $e) {
    $db->rollBack();
    throw $e;
}
}

```

Technical Explanation: Executes complete atomic transaction: locks table context from cryptographic token, validates cart items against database, creates customer profile, calculates grand totals, inserts master order record, and stores itemized snapshots.

Expected System Behavior: Guarantees database consistency through full ACID rollback protection if any component fails during order placement.

Source File: `app/Repositories/OrderRepository.php` | *Module: Order Persistence* | *Reference: Lines 95–130*

Purpose: Frozen snapshot insertion of food dishes and all 8 nutritional macros into `order_items`

```

public function addOrderItem(int $orderId, array $item): int
{
    $sql = "INSERT INTO order_items (
        order_id, food_item_id, food_name_snapshot, base_price_snapshot,
        variant_name_snapshot, variant_price_snapshot, quantity, unit_price, total_price,
        calories, protein, carbs, fat, fiber, sugar, sodium, caffeine
    ) VALUES (
        :order_id, :food_id, :name_snap, :base_snap,
        :var_snap, :var_price, :qty, :unit_price, :total_price,
        :cal, :pro, :carb, :fat, :fib, :sug, :sod, :caff
    )";
    $stmt = $this->db->prepare($sql);
    $stmt->execute([
        'order_id' => $orderId,
        'food_id' => $item['food']['id'],
        'name_snap' => $item['food']['name'],
        'base_snap' => $item['food']['base_price'],
        'var_snap' => $item['variant']['name'] ?? null,
        'var_price' => $item['variant']['price_adjustment'] ?? 0.00,
        'qty' => $item['quantity'],
        'unit_price' => $item['unit_price'],
    ]);
}

```

```

        ':total_price' => $item['line_total'],
        ':cal'      => $item['nutrition']['calories'] ?? null,
        ':pro'      => $item['nutrition']['protein'] ?? null,
        ':carb'     => $item['nutrition']['carbs'] ?? null,
        ':fat'      => $item['nutrition']['fat'] ?? null,
        ':fib'      => $item['nutrition']['fiber'] ?? null,
        ':sug'      => $item['nutrition']['sugar'] ?? null,
        ':sod'      => $item['nutrition']['sodium'] ?? null,
        ':caff'     => $item['nutrition']['caffeine'] ?? null,
    ));
    return (int)$this->db->lastInsertId();
}

```

Technical Explanation: Freezes all dish attributes, names, prices, and 8 nutrient values at the exact moment of order placement.

Expected System Behavior: Future changes to restaurant menu prices or recipes do not alter past historical financial records or customer order receipts.

Source File: `app/Services/PaymentService.php` | *Module: Financial Settlement* | *Reference: Lines 24–65*

Purpose: Realistic payment simulation across Cash, UPI, and Card advancing order lifecycle to accepted

```

public function simulatePayment(int $orderId, string $method): array
{
    $allowed = ['cash', 'upi', 'card'];
    if (!in_array($method, $allowed, true)) throw new \InvalidArgumentException('Invalid payment method.');
```

\$order = \$this->orderRepo->findById(\$orderId);
 if (!\$order) throw new \Exception('Order not found.');

\$txRef = 'TXN-' . strtoupper(\$method) . '-' . strtoupper(substr(md5(uniqid()), 0, 10));

\$paymentId = \$this->paymentRepo->createPayment([
 'order_id' => \$orderId,
 'payment_method' => \$method,
 'transaction_reference' => \$txRef,
 'amount' => \$order['total_amount'],
 'status' => 'completed',
 'paid_at' => date('Y-m-d H:i:s')
]);

\$this->orderRepo->updatePaymentStatus(\$orderId, 'completed');
 \$this->orderRepo->updateStatus(\$orderId, 'accepted');

return ['success' => true, 'transaction_reference' => \$txRef, 'status' => 'completed'];
 }

Technical Explanation: Simulates payment clearing, records a transaction in payments with unique reference, updates order payment status to completed, and advances kitchen lifecycle to accepted.

Expected System Behavior: Eliminates payment friction while maintaining strict referential integrity with a unique 1-to-1 foreign key linkage to orders.

Source File: `app/Controllers/Owner/OrderController.php` | *Module: Kitchen Operations* | *Reference: Lines 75–118*

Purpose: 5-column operational Kanban board status transitions and order card advancement

```

public function updateStatus(int $id): void
{
    $session = RestaurantMiddleware::handle();
    $restaurantId = $session['restaurant_id'];

    $nextStatus = trim((string)Request::post('status', ''));
    $validTransitions = [
        'placed'  => ['accepted', 'cancelled'],
        'accepted' => ['preparing', 'cancelled'],
        'preparing' => ['ready'],
    ];
}

```

```
    'ready' => ['completed'],
  ];

  $order = $this->orderRepo->findWithItemsById($id);
  if (!$order || (int)$order['restaurant_id'] !== $restaurantId) {
    Response::json(['success' => false, 'error' => 'Order not found'], 404);
    return;
  }

  $currentStatus = $order['order_status'];
  if (!isset($validTransitions[$currentStatus]) || !in_array($nextStatus, $validTransitions[$currentStatus], true)) {
    Response::json(['success' => false, 'error' => 'Invalid status transition'], 400);
    return;
  }

  $this->orderRepo->updateStatusForRestaurant($id, $restaurantId, $nextStatus);
  Response::json(['success' => true, 'current_status' => $nextStatus]);
}
```

Technical Explanation: Validates state transitions against an enforced state machine matrix. Prevents out-of-order transitions and ensures kitchen staff cannot transition an order from 'placed' directly to 'completed'.

Expected System Behavior: Maintains sequential kitchen workflow and updates the live tracking timeline for the seated customer in real time.

Source File: `app/Repositories/ReviewRepository.php` | *Module: Customer Feedback* | *Reference: Lines 42–72*

Purpose: Customer review management and official restaurant owner response reply persistence

```
public function addReply(int $reviewId, int $restaurantId, string $reply): bool
{
    $sql = "UPDATE reviews
        SET restaurant_reply = :reply,
            replied_at = NOW(),
            updated_at = NOW()
        WHERE id = :review_id AND restaurant_id = :restaurant_id";
    $stmt = $this->db->prepare($sql);
    return $stmt->execute([
        'reply' => $reply,
        'review_id' => $reviewId,
        'restaurant_id' => $restaurantId
    ]);
}
```

Technical Explanation: Updates reviews record by storing the official restaurant owner reply text and stamping the replied_at timestamp, scoped strictly to the restaurant owner's tenant ID.

Expected System Behavior: Enables two-way customer communication, displaying verified owner response badges directly under customer feedback cards.

Source File: `public/assets/js/live-kitchen.js` | *Module: Real-time Tracking* | *Reference: Lines 15–68*

Purpose: Client-side polling engine updating customer 5-stage progress timeline without page reload

```
async function pollOrderStatus() {
  try {
    const resp = await fetch(`/api/orders/table-latest?table_id=${tableId}&restaurant_id=${restaurantId}`);
    const data = await resp.json();
    if (!data || !data.order_status) return;

    const stages = ['placed', 'accepted', 'preparing', 'ready', 'completed'];
    const currentIdx = stages.indexOf(data.order_status);

    stages.forEach((stage, idx) => {
      const el = document.getElementById(`step-${stage}`);
      if (!el) return;
      el.classList.remove('active', 'completed', 'pending');
      if (idx < currentIdx) {
```

```
        el.classList.add('completed');
    } else if (idx === currentIdx) {
        el.classList.add('active');
    } else {
        el.classList.add('pending');
    }
    });
} catch (e) {
    console.error('Tracking poll error:', e);
}
}
setInterval(pollOrderStatus, 5000);
```

Technical Explanation: Executes asynchronous HTTP polling every 5 seconds against the latest table order endpoint. Updates the CSS classes of 5 timeline DOM nodes to reflect real-time kitchen progress.

Expected System Behavior: Provides seated diners with immediate visual feedback as their dishes are accepted, prepared, and plated by the kitchen.

Source File: [app/Controllers/Owner/TableController.php](#) | *Module: Table & QR Infrastructure* | *Reference: Lines 35–80*

Purpose: Sequential table management (1–12), cryptographic QR generation, and real-time occupancy toggling

```
public function updateStatus(int $id): void
{
    $session = RestaurantMiddleware::handle();
    $restaurantId = $session['restaurant_id'];

    $newStatus = trim((string)Request::post('status', 'available'));
    $allowed = ['available', 'occupied', 'cleaning', 'out_of_service'];

    if (!in_array($newStatus, $allowed, true)) {
        Response::json(['success' => false, 'error' => 'Invalid table status'], 400);
        return;
    }

    $updated = $this->branchRepo->updateTableStatus($id, $restaurantId, $newStatus);
    Response::json(['success' => $updated, 'new_status' => $newStatus]);
}
```

Technical Explanation: Enables restaurant hosts to click occupancy status pills on Tables 1–12 to immediately toggle status between 'available' and 'occupied' via AJAX.

Expected System Behavior: Updates physical dining room occupancy status in real time without refreshing the management portal view.

Source File: [app/Controllers/Admin/RestaurantController.php](#) | *Module: Multi-Tenant Governance* | *Reference: Lines 45–90*

Purpose: Super Admin restaurant lifecycle governance (Approved, Pending, Suspended) and deep portal inspection

```
public function updateStatus(int $id): void
{
    AdminMiddleware::handle();
    $status = trim((string)Request::post('status', ''));
    $allowed = ['pending', 'approved', 'suspended'];

    if (!in_array($status, $allowed, true)) {
        Session::setFlash('error', 'Invalid restaurant status specified. ');
        Response::redirect('/admin/restaurants');
        return;
    }

    $this->restaurantRepo->updateStatus($id, $status);
    Session::setFlash('success', "Restaurant tenancy status updated to {$status}.");
    Response::redirect('/admin/restaurants');
}
```

Technical Explanation: Allows the Platform Super Admin to inspect restaurant tenant details, audit active branches, review food menus, and approve or suspend tenant access.

Expected System Behavior: Approved restaurants immediately go live for customer QR ordering; suspended restaurants are immediately blocked from public access.

Source File: `app/Core/Validator.php` | *Module: Data Integrity* | *Reference: Lines 18–60*

Purpose: Multi-rule server-side input sanitization, type verification, and error aggregation

```
public function validate(array $data, array $rules): bool
{
    $this->errors = [];
    foreach ($rules as $field => $fieldRules) {
        $val = $data[$field] ?? null;
        foreach ($fieldRules as $rule) {
            if ($rule === 'required' && ($val === null || trim((string)$val) === '')) {
                $this->errors[$field][] = "The {$field} field is required.";
            } elseif ($rule === 'email' && $val && !filter_var($val, FILTER_VALIDATE_EMAIL)) {
                $this->errors[$field][] = "The {$field} must be a valid email address.";
            } elseif ($rule === 'numeric' && $val !== null && !is_numeric($val)) {
                $this->errors[$field][] = "The {$field} must be numeric.";
            }
        }
    }
    return empty($this->errors);
}
```

Technical Explanation: Enforces strict validation rules on all server-side form submissions across owner registration, menu creation, customer checkout, and profile updates.

Expected System Behavior: Prevents malformed data from reaching the repository layer and provides clear error feedback to users.

9. TESTING

9.1 TEST PLAN

The test plan was prepared to verify the major functions and technical controls of the Healthy Bite digital restaurant menu and food ordering system. It covers the customer QR ordering journey, food catalog and customization, price and nutrition calculation, cart and checkout validation, order and payment processing, kitchen status management, live order tracking, restaurant owner functions, Super Admin functions, database integrity, authentication and authorization, security controls, responsive and browser-facing behavior, dynamic flows, and release-level validation.

The completed master QA cycle executed 74 test cases against the running PHP application, live MySQL database, application services, routes, and browser-facing functionality. The final result was 74/74 tests passed, with four defects identified, corrected, and retested before release validation.

9.2 TESTING APPROACH

The testing approach was derived from the completed Healthy Bite implementation and the final master QA testing report. The test cases were organized using the same academic testing structure used in the Internship Tracker research paper reference: each test case records a Test Case ID, Input / Test Action, Expected Output, Actual Output, and Status. The testing is divided into functional, integration, database, security, end-to-end, dynamic-flow, and release-validation groups so that each major project module is represented clearly.

Testing was performed in the following order: environment and static checks, database verification, unit/business-logic checks, API validation, authentication and role-based access control, customer end-to-end workflow, owner and kitchen workflow, Super Admin workflow, defensive security testing, dynamic flow testing, catalog/media checks, and final release/regression checks.

9.3 TESTING SUMMARY

Testing Area	Functionality Covered	Status
Environment & Static Analysis	PHP environment, PDO driver, configuration, PHP lint, JavaScript compilation	Successfully Executed
Database Integrity	17 tables, 208 columns, 26 foreign keys, payment mapping, orphan scan	Successfully Executed
Unit / Business Logic	Pricing, GST, 8-macro nutrition, caffeine, NULL preservation, QR and cart logic	Successfully Executed
API Testing	Health, restaurant, categories, foods, cart validation, invalid resource handling	Successfully Executed

Testing Area	Functionality Covered	Status
Authentication & RBAC	Owner/admin protection, invalid credentials, role verification, removed route	Successfully Executed
Customer End-to-End	Order creation, snapshots, simulated payment, status, tracking	Successfully Executed
Owner & Kitchen	Kanban transitions, tables, QR/table status, review replies, analytics	Successfully Executed
Security & Integrity	SQLi, XSS, CSRF, IDOR/tenant isolation, immutable snapshots, concurrency	Successfully Executed
Dynamic Flow / Catalog	Repository dynamic functions, table redirect, media URLs, dietary types	Successfully Executed
Release Validation	HEAD routing, order modal, menu CRUD, legacy feature exclusion	Successfully Executed

9.4 TESTING NOTES

- The test-case tables follow the five-column format: Test Case ID, Input / Test Action, Expected Output, Actual Output, and Status.
- The Actual Output column records the observed and verified behavior from the completed Healthy Bite Master QA validation report; it is not a placeholder result.
- The testing scope includes functional, integration, unit/business-logic, database, security, concurrency, dynamic-flow, browser/API, and release-validation checks.
- Payment is intentionally tested as simulated payment for supported methods. Third-party banking gateway integration is outside the implemented project scope.
- Live customer tracking is tested through the implemented HTTP polling mechanism.
- The legacy admin table is treated as a database artifact and is not the active administrator authentication mechanism.
- All 74 executed test cases passed after four defects were corrected and regression-tested.

9.5 TEST CASES

9.5.1 Customer / Core / API Test Cases

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-01	PHP 8.2+ Compatibility	PHP version is >= 8.2 (Running: PHP 8.5.10)	PHP 8.5.10 detected; compatibility requirement satisfied	Successfully Executed
T-02	PDO MySQL Driver	pdo_mysql extension active and loaded	pdo_mysql loaded successfully	Successfully Executed
T-03	Live Database Connection	PDO connection to healthy_bite responds SELECT 1 = 1	Live database connection responded successfully	Successfully Executed
T-04	.env Configuration	.env present, loaded and parseable via App\Core\Env	.env loaded and parsed successfully	Successfully Executed

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-05	Core Constants	ROLE_*, ORDER_*, TABLE_*, FOOD_* constants defined	Required core constants verified	Successfully Executed
T-06	PHP Syntax Validation	All 94 project PHP files pass php -l lint with zero errors	All 94 PHP files passed php -l	Successfully Executed
T-07	JavaScript VM Compilation	All 14 public JS files pass Node.js VM syntax compilation	All 14 JavaScript files compiled successfully	Successfully Executed
T-08	17 Physical Tables	All 17 documented tables verified in information_schema	All 17 physical tables verified	Successfully Executed
T-09	208 Database Columns	Exact column count across all 17 tables matches 208	208 columns verified across 17 tables	Successfully Executed
T-10	26 Foreign Keys	All 26 foreign key constraints verified active	All 26 foreign keys active and verified	Successfully Executed
T-11	Order-Payment Mapping 1:1	payments.order_id UNIQUE key strictly enforced (Non_unique=0)	Unique order-to-payment mapping verified	Successfully Executed
T-12	Orphan Record Scan	Zero orphan rows across all 26 FK relations	0 orphan rows found	Successfully Executed
T-13	Base + Variant + Custom Price	Base (100) + Var (20) + Custom (25) = Rs 145.00	Rs 145.00 calculated exactly	Successfully Executed
T-14	Line Total Scaling	Rs 145.00 x 3 = Rs 435.00 exact line total	Rs 435.00 line total calculated exactly	Successfully Executed
T-15	Order Totals & 5% GST	Subtotal Rs 435 + Tax Rs 21.75 = Rs 456.75	Rs 456.75 final total verified	Successfully Executed
T-16	8 Macros & Caffeine Sum	All 8 macros summed: Cal, Prot, Carb, Fat, Fib, Sug, Sod, Caff	All 8 nutrition values summed correctly	Successfully Executed
T-17	Caffeine in mg	Caffeine calculated in mg and never coerced to grams	Caffeine remained in mg throughout calculation	Successfully Executed
T-18	Quantity Nutrition Scaling	Nutrition doubles accurately when quantity = 2	Nutrition doubled accurately for quantity 2	Successfully Executed
T-19	Strict NULL Preservation	Unmeasured fields retain NULL, not coerced to 0	NULL nutrition fields remained NULL	Successfully Executed
T-20	Valid QR Token Resolution	Active token resolves restaurant, branch, and table IDs	Active QR token resolved correct context	Successfully Executed
T-21	Invalid QR Token Rejection	Random/expired token returns null context	Invalid or expired token returned null context	Successfully Executed
T-22	Empty Cart Rejection	Empty cart payload throws InvalidArgumentException	Empty cart rejected correctly	Successfully Executed
T-23	Cross-Restaurant Isolation	Food from Restaurant A rejected under Restaurant B context	Cross-restaurant food access blocked	Successfully Executed
T-24	GET /api/health	Returns HTTP 200 with status=ok and database=connected	HTTP 200 and healthy DB status returned	Successfully Executed
T-25	GET /api/restaurant	Returns HTTP 200 with active restaurant profile	Active restaurant profile returned	Successfully Executed
T-26	GET /api/categories	Returns HTTP 200 with list of active menu categories	Active category list returned	Successfully Executed

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-27	GET /api/foods	Returns HTTP 200 with complete menu items catalog	Complete menu catalog returned	Successfully Executed
T-28	GET /api/foods/1	Returns HTTP 200 with Paneer Bowl details & variants	Paneer Bowl details and variants returned	Successfully Executed
T-29	GET /api/foods/99999	Returns HTTP 404 for non-existent food item ID	HTTP 404 returned correctly	Successfully Executed
T-30	POST /api/cart/validate	Returns HTTP 200 for valid items & required groups	Valid cart returned HTTP 200 with verified totals	Successfully Executed
T-31	Cart Server Recalculation	Server ignores client fake prices; computes from DB	Fake prices ignored; authoritative DB values used	Successfully Executed
T-32	Owner Dashboard Protection	Unauthenticated /owner/dashboard redirects to /owner/login	Unauthenticated access redirected to owner login	Successfully Executed
T-33	Admin Dashboard Protection	Unauthenticated /admin/dashboard redirects to /admin/login	Unauthenticated access redirected to admin login	Successfully Executed
T-34	Invalid Credentials Check	Bad email/password fails authentication gracefully	Invalid credentials rejected cleanly	Successfully Executed
T-35	Removed Admin Orders Route	/admin/orders returns HTTP 404 (strictly eliminated)	HTTP 404 confirmed; route absent	Successfully Executed
T-36	Order Creation Pipeline	Order placed with valid customer, table, and branch	Order created successfully with valid context	Successfully Executed
T-37	Order Item Snapshot Freeze	food_name, base_price, unit_price saved in snapshot	Historical item snapshot persisted correctly	Successfully Executed
T-38	Customization Snapshot Freeze	customization_name and price_adjustment saved in snapshot	Customization snapshot persisted correctly	Successfully Executed
T-39	Payment Simulation (UPI)	Simulates payment, generates TXN ref, marks completed	UPI simulation completed with transaction reference	Successfully Executed
T-40	Status Advance on Payment	Order status advances from placed to accepted upon payment	Order advanced to accepted after payment	Successfully Executed
T-41	Order Tracking Polling	Live tracking finds order by order_number with status	Live tracking returned current order status	Successfully Executed

9.5.2 Owner / Kitchen / Admin Test Cases

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-42	Kitchen Kanban State Machine	placed->accepted->preparing->ready->completed	All required status transitions succeeded	Successfully Executed
T-43	Dining Tables 1-12 Active	All 12 dining tables present and queryable for Branch 1	Tables 1-12 verified and queryable	Successfully Executed
T-44	Table Status Toggle	Table toggles available <-> occupied seamlessly	Table status toggled successfully	Successfully Executed
T-45	Review Reply Persistence	Owner reply persisted to reviews table with replied_at	Reply persisted with replied_at timestamp	Successfully Executed
T-46	Super Admin Role Verification	Super Admin account verified with role_id = 1	Super Admin role verified	Successfully Executed

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-47	Restaurant Owner Role Verification	Owner account verified with role_id = 2	Restaurant Owner role verified	Successfully Executed
T-48	Kitchen Staff Role Verification	Kitchen Staff account verified with role_id = 4	Kitchen Staff role verified	Successfully Executed
T-49	Approved Restaurants Query	Platform lists approved active tenant restaurants	Approved tenant restaurants listed correctly	Successfully Executed

9.5.3 Security / Integrity / Concurrency Test Cases

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-50	SQL Injection: Search Filter	Malicious SQL in search handled via PDO prepared statements	Malicious SQL handled safely	Successfully Executed
T-51	SQL Injection: Query Params	IDs and filters parameterized; zero SQL injection vectors	IDs and filters safely parameterized	Successfully Executed
T-52	XSS Output Escaping	HTML/JS tags escaped via htmlspecialchars in views	Output escaping verified	Successfully Executed
T-53	CSRF Protection	State-changing POST actions protected by CSRF tokens	CSRF validation blocked unauthorized state changes	Successfully Executed
T-54	IDOR Tenant Protection	Cross-restaurant cart and order modification blocked	Cross-tenant modification blocked	Successfully Executed
T-55	Catalog Price Independence	Updating food base_price does not alter historical order snapshot	Historical order snapshot remained unchanged	Successfully Executed
T-56	Duplicate Payment Guard	Repeated payment request on paid order blocked gracefully	Repeated payment rejected cleanly	Successfully Executed

9.5.4 Dynamic Flow / Catalog / Release Test Cases

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-57	Find Table with Active Token	BranchRepository::findTableByNumber locates table + token	Table and token located correctly	Successfully Executed
T-58	Find First Available Table	BranchRepository::getFirstAvailableTable returns active table	Active table returned correctly	Successfully Executed
T-59	Table Number Redirect	HTTP GET /table/12 redirects (302) to /menu?token=...	HTTP 302 redirect verified	Successfully Executed
T-60	Invalid Token Error Banner	HTTP GET /menu with invalid token displays error notice	Error notice displayed correctly	Successfully Executed
T-61	Category Auto-Slug Generation	CategoryRepository::createCategory generates slug automatically	Slug generated automatically	Successfully Executed
T-62	Category Deletion Guard	Category deletion blocked if active food items are attached	Deletion guard blocked invalid removal	Successfully Executed
T-63	Staff Record Update	StaffRepository::updateStaff dynamically updates member details	Staff details updated successfully	Successfully Executed

Test Case ID	Input / Test Action	Expected Output	Actual Output	Status
T-64	Macro Aggregations Sugar	OrderRepository::getMacroStats computes total & avg sugar	Sugar totals and averages computed correctly	Successfully Executed
T-65	Checkout Dropdown Table	Checkout view renders verified active branch tables	Active branch tables rendered correctly	Successfully Executed
T-66	Owner Analytics Data Load	Analytics controller loads real AOV, revenue, orders	Real analytics data loaded successfully	Successfully Executed
T-67	Sugar Snapshot Integrity	OrderService creates order with accurate sugar snapshot	Sugar snapshot verified accurate	Successfully Executed
T-68	45 Unique Food Dish URLs	All 45 Unsplash dish image URLs return HTTP 200 OK	All 45 URLs returned HTTP 200	Successfully Executed
T-69	Distinct Identifiers Photo	All 45 food items use 100% distinct photo IDs	All 45 food items use distinct photo IDs	Successfully Executed
T-70	Food Dietary Types Valid	food_type enum values strictly vegetarian, vegan, etc.	Valid food_type enum values confirmed	Successfully Executed
T-71	HTTP HEAD Method Route Matching	Router maps HEAD to GET; curl -I returns HTTP 200 OK	HEAD request returned HTTP 200	Successfully Executed
T-72	Order Details Modal Retrieval	Order modal API retrieves items and customizations	Order details and customizations retrieved	Successfully Executed
T-73	Menu Item Create/Edit/Delete	Full food item CRUD lifecycle tested and restored	Full CRUD lifecycle passed	Successfully Executed
T-74	Legacy Features Clean Exclusion	No Call Servant or Platform Orders code exists	Removed legacy features remained absent	Successfully Executed

9.5.5 DEFENSIVE SECURITY TESTING RESULTS

Attack Vector / Scope	Security Test Applied	Protection Mechanism	Result
SQL Injection (SQLi)	Injected malicious SQL into search, query parameters and IDs.	PDO prepared statements with parameterized placeholders across repositories.	PASS
Cross-Site Scripting (XSS)	Injected script-like payloads into names, notes and customer review comments.	htmlspecialchars output escaping with ENT_QUOTES and UTF-8 in views.	PASS
Cross-Site Request Forgery	Dispatched state-changing POST requests without valid CSRF session tokens.	Csrf::validate() token verification rejects unauthorized state changes.	PASS
IDOR & Multi-Tenant Isolation	Attempted to order Food from Restaurant 1 under Restaurant 2 context via API payload.	CartService and SQL queries enforce strict restaurant_id multi-tenant scoping.	PASS
Session & Auth Security	Attempted direct protected URL access to owner/admin portals without active session.	RestaurantMiddleware and AdminMiddleware redirect unauthenticated sessions to login.	PASS

9.5.6 VERIFIED DATABASE SCHEMA ARCHITECTURE

Table Name	Rows	Cols	FKs	Primary Key	Role / Constraints
admin	4	4	0	id	Legacy static administration table
branches	6	10	1	id	Restaurant dining locations
categories	16	10	1	id	Menu category hierarchy
customers	37	6	0	id	Registered and guest patrons
food_customizations	564	21	1	id	Dish modifiers and adjustments
food_items	47	25	2	id	Main culinary catalog and macros
food_variants	94	18	1	id	Portion sizes and variant adjustments
order_item_customizations	66	15	2	id	Frozen customization snapshot
order_items	43	19	2	id	Frozen item price/macro snapshot
orders	34	16	4	id	Core order transactions and statuses
payments	21	8	1	id	Tender settlement (UNIQUE: order_id)
qr_tokens	12	8	3	id	Table dining session tokens
restaurant_tables	12	7	2	id	Physical dining tables (1-12)
restaurants	3	15	1	id	Tenant restaurant accounts
reviews	7	11	3	id	Customer ratings and owner replies
roles	4	6	0	id	RBAC roles (admin, owner, manager, staff)
users	8	9	2	id	Authenticated employee and admin logins

9.6 FINAL TESTING STATEMENT

The Healthy Bite system was tested across its major customer, owner, kitchen, administrator, database, API, security, and release-validation functions. A total of 74 test cases were executed. All 74 test cases passed successfully after four identified defects were corrected and regression-tested. The final testing cycle found no unresolved critical defects, and the application was verified for final academic submission and release within the implemented project scope.

Test Execution	74 / 74 Passed (100%)
Defects	4 Identified, Fixed and Retested
Regression	All related suites passed after fixes
Final Status	VERIFIED & APPROVED

10. RESULTS

Results Summary Table

Result Area	Functional Scope Evaluated	Observed Empirical Outcome	Validation Status
Customer Onboarding	Table QR resolution, fallback routing	Session created instantly; invalid tokens gracefully route to default menu	100% Passed
Menu & Dietary	Categories, dietary filters, search	Dynamic filtering by Veg/Vegan/Non-Veg/Jain; sub-100ms keyword search	100% Passed
Nutritional Transparency	8-macro display, allergen disclosure	Accurate calories, protein, carbs, fat, fiber, sugar, sodium, caffeine display	100% Passed
Cart & Customization	Variant selection, ingredient delta, edit	Portion scaling and in-cart customization editing recalculate price/macros	100% Passed
Order & Payment	Unique order generation, 5% GST, COD/simulated	Alphanumeric IDs created; mathematical tax computation verified	100% Passed
Live Kitchen Tracking	5-stage pipeline, background polling	State changes propagate to diner view within 4s without full-page reloads	100% Passed
Kitchen Operations	Kanban board, rapid order status transitions	Single-click state advance (Placed -> Accepted -> Preparing -> Ready)	100% Passed
Owner Administration	Menu CRUD, slug generation, analytics	Automated slugging, media format validation, real-time KPI data loading	100% Passed
Database Integrity	17 tables, 208 columns, 26 foreign keys	Zero orphan records detected; historical order snapshots preserved	100% Passed
Defensive Security	SQLi, XSS, CSRF, IDOR, RBAC	Parameterized queries, output encoding, CSRF tokens, strict tenant isolation	100% Passed

11. CONCLUSION

11.1 Research and Implementation Synthesis

Modern online food ordering systems have achieved widespread commercial adoption by maximizing ordering convenience and delivery velocity. However, as established in the problem statement, this convenience has historically come at the expense of nutritional transparency. Mainstream platforms prioritize promotional discounts and rapid dispatch while treating food as a black-box commodity, offering negligible disclosure of calories, macronutrients, allergens, or ingredient origins. This information asymmetry directly undermines consumer health awareness and presents severe challenges for health-conscious diners, fitness enthusiasts, and individuals managing dietary conditions such as diabetes, obesity, and hypertension.

Healthy Bite: Digital Menu and Food Ordering System was conceived, designed, and developed to bridge this critical gap between digital food ordering convenience and proactive health management. The research and engineering effort demonstrates that full nutritional transparency can be seamlessly woven into every stage of the restaurant customer journey—from contactless QR table onboarding and ingredient customization to live kitchen tracking—without introducing operational friction for restaurant personnel or computational overhead for consumers.

11.2 Direct Alignment with Project Objectives

The completed implementation directly satisfies every core objective formulated in Section 3 of this research paper:

- **Fulfillment of Objective 1 (Nutritional Transparency):** The platform provides an engaging, responsive digital dining menu featuring 8 verified nutritional metrics per dish, allergen alerts, ingredient listings, and dynamic dietary badges (Vegetarian, Vegan, Non-Vegetarian, Jain-friendly).
- **Fulfillment of Objective 2 (Interactive Meal Customization):** Diners can dynamically modify portion sizes, carb bases, dressings, and protein add-ons, with the client and server recalculating both price and macronutrient totals in real time.
- **Fulfillment of Objective 3 (Contactless Table Ordering):** Physical dining tables are provisioned with dynamic QR tokens, enabling zero-contact onboarding, digital ordering, and instant kitchen ticket generation without waiter intervention.
- **Fulfillment of Objective 4 (Live Kitchen Operations):** The operational Kanban board provides kitchen staff with an intuitive tool to transition orders across 5 verified

stages (Placed, Accepted, Preparing, Ready, Completed), propagating status updates to diner views within 4 seconds.

- **Fulfillment of Objective 5 (Owner Management & Analytics):** Restaurant owners possess full administrative control over menus, categories, ingredients, variants, tables, and customer reviews, supported by real-time analytics on sales and nutritional distribution.
- **Fulfillment of Objective 6 (Data Integrity & Security):** The system enforces robust defense-in-depth security, including PDO prepared statements, output encoding, CSRF tokens, strict multi-tenant isolation, and complete historical snapshot preservation across 17 normalized relational tables.

11.3 Key Architectural and Technological Contributions

The technical execution of Healthy Bite yielded several noteworthy software engineering contributions:

- **High-Performance Lightweight Architecture:** By avoiding heavyweight client frameworks and instead implementing clean Vanilla CSS and modular ES6+ JavaScript, the application achieves rapid sub-second page loads, minimal DOM overhead, and broad cross-device compatibility.
- **Immutable Snapshot Preservation:** The database architecture solves the common industry flaw of retroactive order alteration by taking immutable snapshots of dish names, variant names, customization choices, prices, and macronutrient baselines upon order placement.
- **Secure Multi-Tenant Partitioning:** The platform enforces strict tenant-level relational isolation, ensuring that multiple restaurant branches can co-exist within the same database infrastructure without risk of data cross-contamination or unauthorized access.
- **Operational Transparency:** The live kitchen pipeline bridges the operational gap between back-of-house kitchen preparation and front-of-house diner expectations, reducing perceived waiting times and customer anxiety.

11.4 Future Scope and Research Directions

While the current implementation achieves full functional maturity within its academic scope, several promising extensions have been identified for future research and commercial development. These enhancements are formally designated as Future Work:

1. AI-Driven Personalized Dietary Recommendations: Integrating machine learning models (e.g., collaborative filtering and content-based recommendation algorithms) to analyze past customer dietary choices and recommend personalized meals aligned with specific caloric, keto, diabetic, or athletic goals.

2. Automated BMI & Calorie Budgeting Calculator: Implementing an interactive health profile module allowing users to input height, weight, activity levels, and wellness goals to compute dynamic Basal Metabolic Rates (BMR) and recommended daily caloric thresholds.

3. Wearable Device & Fitness Platform Integration: Integrating health APIs (such as Apple HealthKit, Google Health Connect, and Fitbit Web API) to synchronize consumed meal macros directly with diners' personal fitness trackers.

4. Native Mobile Applications: Developing native mobile applications for Android and iOS using cross-platform frameworks (e.g., Flutter or React Native) to provide offline caching, geofenced table detection, and push notifications.

5. Voice-Assisted Conversational Ordering: Incorporating voice-activated conversational interfaces utilizing Web Speech APIs to facilitate hands-free food ordering and improve accessibility for visually impaired or elderly patrons.

6. Commercial Payment Gateway Integration: Extending the simulated payment layer to integrate licensed commercial payment gateways (such as UPI 2.0, Razorpay, or Stripe), enabling secure automated credit card processing, digital wallets, and automated refunds.

7. Multi-Lingual Regional Localization: Extending language catalogs to support multi-lingual localization across regional Indian and international languages, broadening demographic accessibility